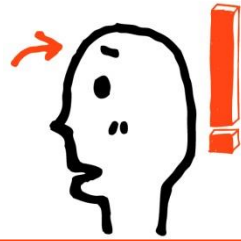




Android Java



자바의 다양한 기본 클래스

- 01** Wrapper 클래스
- 02** BigInteger 클래스와 BigDecimal 클래스
- 03** Math 클래스와 난수의 생성, 그리고 문자열 토큰의 구분



기본 자료형 데이터를 감싸는 Wrapper 클래스

```
public static void showData(Object obj)
{
    System.out.println(obj);
}
```

왼쪽의 메소드에서 보이듯이 기본 자료형 데이터를 인스턴스화 해야 하는 상황이 발생할 수 있다. 이러한 상황에 사용할 수 있는 클래스를 가리켜 **Wrapper** 클래스라 한다.

```
class IntWrapper
{
    private int num;
    public IntWrapper(int data)
    {
        num=data;
    }
    public String toString()
    {
        return ""+num;
    }
}
```

프로그래머가 정의한 **int**형 기본 자료형에 대한 **Wrapper** 클래스!
이렇듯 **Wrapper** 클래스는 기본 자료형 데이터를 **클 저장 및 참조할 수 있는 구조로 정의된다.**



자바에서 제공하는 Wrapper 클래스

• Boolean	Boolean(boolean value)
• Character	Character(char value)
• Byte	Byte(byte value)
• Short	Short(short value)
• Integer	Integer(int value)
• Long	Long(long value)
• Float	Float(float value), Float(double value)

순전히 기본 자료형 데이터의 표현이 목적이라면, 별도의 클래스 정의 없이 제공되는 **Wrapper** 클래스를 사용하면 된다!

위의 클래스 이외에도 문자열 기반으로 정의된 **Wrapper** 클래스도 존재하기 때문에 다음과 같이 인스턴스 생성도 가능. 단, **Character** 클래스 제외!

```
Integer num1=new Integer("240")
Double num2=new Double("12.257");
```

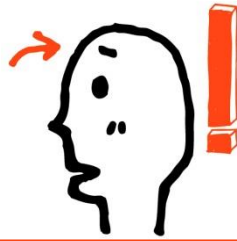
자바제공 Wrapper 클래스 사용의 예!

```
public static void main(String[] args)
{
    Integer intInst=new Integer(3);
    showData(intInst);
    showData(new Integer(7));
}
```



주요 메소드

메소드	설명
<code>static int bitCount(int i)</code>	인자 <code>i</code> 의 이진수 표현에서 1의 개수를 리턴
<code>float floatValue()</code>	<code>float</code> 타입으로 변환된 값 리턴
<code>int intValue()</code>	<code>int</code> 타입으로 변환된 값 리턴
<code>long longValue()</code>	<code>long</code> 타입으로 변환된 값 리턴
<code>short shortValue()</code>	<code>short</code> 타입으로 변환된 값 리턴
<code>static int parseInt(String s)</code>	스트링 <code>s</code> 를 10진 정수로 변환된 값 리턴
<code>static int parseInt(String s, int radix)</code>	스트링 <code>s</code> 를 지정된 진법의 정수로 변환된 값 리턴
<code>static String toBinaryString(int i)</code>	인자 <code>i</code> 를 이진수 표현으로 변환된 스트링 리턴
<code>static String toHexString(int i)</code>	인자 <code>i</code> 를 16진수 표현으로 변환된 스트링 리턴
<code>static String toOctalString(int i)</code>	인자 <code>i</code> 를 8진수 표현으로 변환된 스트링 리턴
<code>static String toString(int i)</code>	인자 <code>i</code> 를 스트링으로 변환하여 리턴



Wrapper 활용

□ Wrapper 객체로부터 기본 데이터 타입 알아내기

```
Integer i = new Integer(10);  
int ii = i.intValue(); // ii = 10
```

```
Character c = new Character('c');  
char cc = c.charValue(); // cc = 'c'
```

```
Float f = new Float(3.14);  
float ff = f.floatValue(); // ff = 3.14
```

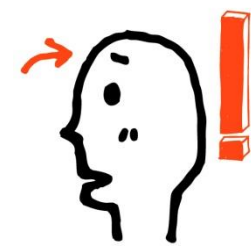
```
Boolean b = new Boolean(true);  
boolean bb = b.booleanValue(); // bb = true
```

□ 문자열을 기본 데이터 타입으로 변환

```
int i = Integer.parseInt("123"); // i = 123  
boolean b = Boolean.parseBoolean("true"); // b = true  
float f = Float.parseFloat("3.141592"); // f = 3.141592
```

□ 기본 데이터 타입을 문자열로 변환

```
String s1 = Integer.toString(123); // 정수 123을 문자열 "123" 으로 변환  
String s3 = Float.toString(3.141592f); // 실수 3.141592를 문자열 "3.141592"로 변환  
String s4 = Character.toString('a'); // 문자 'a'를 문자열 "a"로 변환  
String s5 = Boolean.toString(true); // 불린 값 true를 문자열 "true"로 변환
```



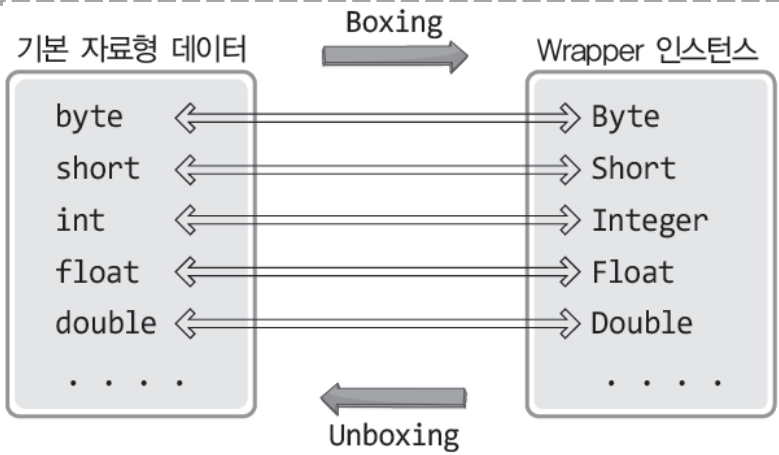
Wrapper 클래스의 두 가지 기능

Boxing

➔ 기본 자료형 데이터를
Wrapper 인스턴스로 감싸는 것!

UnBoxing

➔ Wrapper 인스턴스에 저장된 데이터를
꺼내는 것!



```
class BoxingUnboxing
{
    public static void main(String[] args)
    {
        Integer iValue=new Integer(10);
        Double dValue=new Double(3.14);
        System.out.println(iValue);
        System.out.println(dValue);

        iValue=new Integer(iValue.intValue()+10);
        dValue=new Double(dValue.doubleValue()+1.2);
        System.out.println(iValue);
        System.out.println(dValue);
    }
}
```

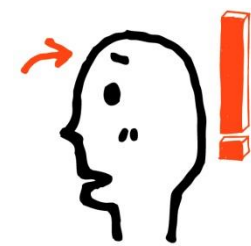
실행 결과

10
3.14
20
4.34

본래 Wrapper 클래스는 산술연산을 고려
해서 정의되는 클래스가 아니다. 따라서
Wrapper 인스턴스를 대상으로 산술연산
을 할 경우에는 왼쪽과 같이 코드가 복잡
해진다.



Auto Boxing & Auto Unboxing
을 이용하면 다소 편해진다!

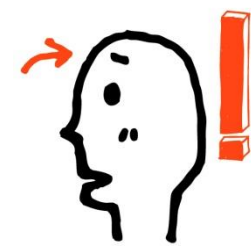


주요 메소드

■ 언박싱 코드

- 각 포장 클래스마다 가지고 있는 클래스 호출
- 기본 타입명 + Value()

기본 타입의 값을 이용		
byte	num	= obj.byteValue();
char	ch	= obj.charValue();
short	num	= obj.shortValue();
int	num	= obj.intValue();
long	num	= obj.longValue();
float	num	= obj.floatValue();
double	num	= obj.doubleValue();
boolean	bool	= obj.booleanValue();



Auto Boxing & Auto Unboxing

자바제공 Wrapper 클래스를 사용하는 것이 좋은 이유!

Auto Boxing

→ 기본 자료형 데이터가 자동으로 Wrapper 인스턴스로 감싸지는 것!

Auto UnBoxing

→ Wrapper 인스턴스에 저장된 데이터가 자동으로 꺼내지는 것!

기본 자료형 데이터와 와야 하는데, Wrapper 인스턴스가 있다면, Auto Unboxing!

인스턴스가 와야 하는데, 기본 자료형 데이터가 있다면, Auto Boxing!

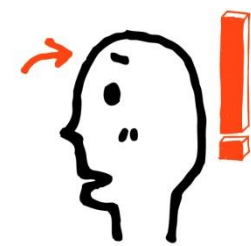
```
public static void main(String[] args)
{
    Integer iValue=10;        // auto boxing
    Double dValue=3.14;       // auto boxing

    System.out.println(iValue);
    System.out.println(dValue);

    int num1=iValue;          // auto unboxing
    double num2=dValue;       // auto unboxing
    System.out.println(num1);
    System.out.println(num2);
}
```

실행 결과

10
3.14
10
3.14



Auto Boxing & Auto Unboxing 어떻게?

```
public static void main(String[] args)
{
    Integer num1=10;
    Integer num2=20;

    num1++;
    System.out.println(num1);

    num2+=3;
    System.out.println(num2);

    int addResult=num1+num2;
    System.out.println(addResult);

    int minResult=num1-num2;
    System.out.println(minResult);
}
```

Auto Boxing, Auto Unboxing 동시 발생

num1=new Integer(num1.intValue()+1);

Auto Boxing, Auto Unboxing 동시 발생

num2=new Integer(num2.intValue()+3);

11
23
34
-12

실행 결과

예제에서 보이듯이 Auto Boxing과 Unboxing은 다양한 형태로 진행된다. 특히 산술연산의 과정에서도 발생을 한다는 사실에 주목할 필요가 있다.



Auto Boxing & Auto Unboxing 어떻게?

❖ 문자열을 기본 타입 값으로 변환

- parse + 기본타입 명 → 정적 메소드

기본 타입의 값을 이용		
byte	num	= Byte.parseByte("10");
short	num	= Short.parseShort("100");
int	num	= Integer.parseInt("1000");
long	num	= Long.parseLong("10000");
float	num	= Float.parseFloat("2.5F");
double	num	= Double.parseDouble("3.5");
boolean	bool	= Boolean.parseBoolean("true");

❖ 포장값 비교

- 포장 객체는 내부 값을 비교하기 위해 ==와 != 연산자 사용 불가
- 값을 언박싱해 비교하거나, equals() 메소드로 내부 값 비교할 것

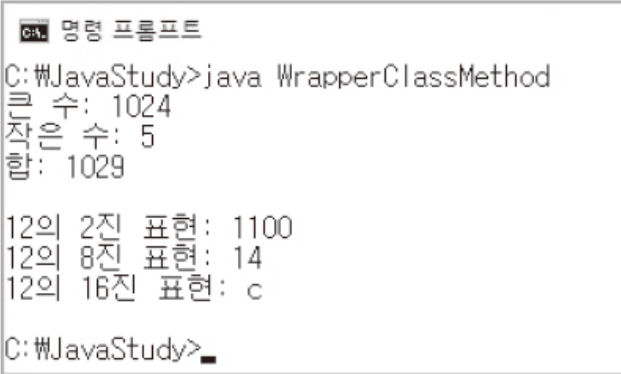


래퍼 클래스의 다양한 static 메소드들

```
public static void main(String[] args) {
    // 클래스 메소드를 통한 인스턴스 생성 방법 두 가지
    Integer n1 = Integer.valueOf(5);    // 숫자 기반 Integer 인스턴스 생성
    Integer n2 = Integer.valueOf("1024"); // 문자열 기반 Integer 인스턴스 생성

    // 대소 비교와 합을 계산하는 클래스 메소드
    System.out.println("큰 수: " + Integer.max(n1, n2));
    System.out.println("작은 수: " + Integer.min(n1, n2));
    System.out.println("합: " + Integer.sum(n1, n2));
    System.out.println();

    // 정수에 대한 2진, 8진, 16진수 표현 결과를 반환하는 클래스 메소드
    System.out.println("12의 2진 표현: " + Integer.toBinaryString(12));
    System.out.println("12의 8진 표현: " + Integer.toOctalString(12));
    System.out.println("12의 16진 표현: " + Integer.toHexString(12));
}
```



매우 큰 정수의 표현을 위한 BigInteger 클래스

```
public static void main(String[] args) {
    // long형으로 표현 가능한 값의 크기 출력
    System.out.println("최대 정수: " + Long.MAX_VALUE);
    System.out.println("최소 정수: " + Long.MIN_VALUE);
    System.out.println();

    // 매우 큰 수를 BigInteger 인스턴스로 표현
    BigInteger big1 = new BigInteger("10000000000000000000000");
    BigInteger big2 = new BigInteger("-9999999999999999999999");

    // BigInteger 기반 덧셈 연산
    BigInteger r1 = big1.add(big2);
    System.out.println("덧셈 결과: " + r1);

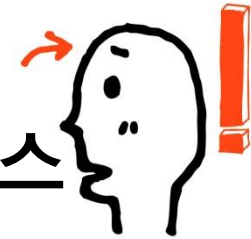
    // BigInteger 기반 곱셈 연산
    BigInteger r2 = big1.multiply(big2);
    System.out.println("곱셈 결과: " + r2);
    System.out.println();

    // 인스턴스에 저장된 값을 int형 정수로 반환
    int num = r1.intValueExact();
    System.out.println("From BigInteger: " + num); }

```

큰 정수 n 을 문자열로 표현한 이유는 숫자로 표현이 불가능하기 때문이다! 기본 자료형의 범위를 넘어서는 크기의 정수는 숫자로 표현 불가능하다!

[illegible]



오차 없는 실수의 표현을 위한 BigDecimal 클래스

```
public static void main(String[] args)
{
    BigDecimal e1=new BigDecimal(1.6);
    BigDecimal e2=new BigDecimal(0.1);

    System.out.println("두 실수의 덧셈결과 : "+ e1.add(e2));
    System.out.println("두 실수의 곱셈결과 : "+ e1.multiply(e2));
}
```

실수 1.6과 0.1을 숫자로 표현하는 순간
이미 오차가 포함되어 버렸다.

실행 결과

1.89999999999999996669330926124530378
0.340000000000000000999200722162640837

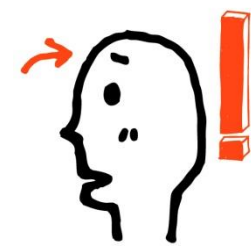
```
public static void main(String[] args)
{
    BigDecimal e1=new BigDecimal("1.6");
    BigDecimal e2=new BigDecimal("0.1");

    System.out.println("두 실수의 덧셈결과 : "+ e1.add(e2));
    System.out.println("두 실수의 곱셈결과 : "+ e1.multiply(e2));
}
```

실수도 문자열로 표현을 해서, BigDecimal
클래스에 오차 없는 값을 전달해야 한다!

실행 결과

두 실수의 덧셈결과 : 1.7
두 실수의 곱셈결과 : 0.16



수학관련 기능을 제공하는 Math 클래스

```
public static void main(String[] args)
{
    System.out.println("원주율 : " + Math.PI);
    System.out.println("2의 제곱근 : " + Math.sqrt(2));

    System.out.println(
        "파이에 대한 Degree : " + Math.toDegrees(Math.PI));
    System.out.println(
        "2파이에 대한 Degree : " + Math.toDegrees(2.0*Math.PI));

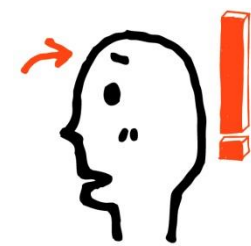
    double radian45=Math.toRadians(45);    // 라디안으로의 변환!
    System.out.println("싸인 45 : " + Math.sin(radian45));
    System.out.println("코싸인 45 : " + Math.cos(radian45));
    System.out.println("탄젠트 45 : " + Math.tan(radian45));

    System.out.println("로그 25 : " + Math.log(25));
    System.out.println("2의 4승 : "+ Math.pow(2, 4));
}
```

실행 결과

원주율 : 3.141592653589793
2의 제곱근 : 1.4142135623730951
파이에 대한 Degree : 180.0
2파이에 대한 Degree : 360.0
싸인 45 : 0.7071067811865475
코싸인 45 : 0.7071067811865476
탄젠트 45 : 0.9999999999999999
로그 25 : 3.2188758248682006
2의 4승 : 16.0

Math 클래스에는 수학관련 메소드가 static으로 정의되어 있다는 사실을 기억하는 것이 중요하다!
필요한 메소드는 API 문자를 통해 참조하면 된다. 단 대부분의 메소드가 라디안 단위로 정의되어
있음은 기억하기 바란다!

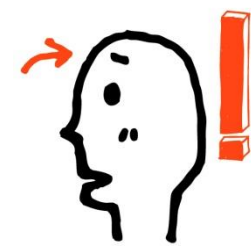


수학관련 기능을 제공하는 Math 클래스

❖ Math 클래스

■ 수학 계산에 사용할 수 있는 정적 메소드 제공

메소드	설명	예제 코드	리턴값
int abs(int a) double abs(double a)	절대값	int v1 = Math.abs(-5); double v2 = Math.abs(-3.14);	v1 = 5 v2 = 3.14
double ceil(double a)	올림값	double v3 = Math.ceil(5.3); double v4 = Math.ceil(-5.3);	v3 = 6.0 v4 = -5.0
double floor(double a)	버림값	double v5 = Math.floor(5.3); double v6 = Math.floor(-5.3);	v5 = 5.0 v6 = -6.0
int max(int a, int b) double max(double a, double b)	최대값	int v7 = Math.max(5, 9); double v8 = Math.max(5.3, 2.5);	v7 = 9 v8 = 5.3
int min(int a, int b) double min(double a, double b)	최소값	int v9 = Math.min(5, 9); double v10 = Math.min(5.3, 2.5);	v9 = 5 v10 = 2.5
double random()	랜덤값	double v11 = Math.random();	0.0<= v11<1.0
double rint(double a)	가까운 정수의 실수값	double v12 = Math.rint(5.3); double v13 = Math.rint(5.7);	v12 = 5.0 v13 = 6.0
long round(double a)	반올림값	long v14 = Math.round(5.3); long v15 = Math.round(5.7);	v14 = 5 v15 = 6



수학관련 기능을 제공하는 Random 클래스

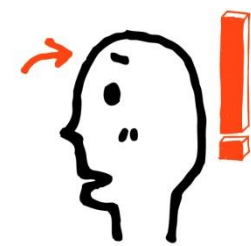
❖ Random 클래스

- boolean, int, long, float, double 난수 입수 가능
- 난수를 만드는 알고리즘에 사용되는 종자값(seed) 설정 가능
 - 종자값이 같으면 같은 난수
- Random 클래스로 부터 Random객체 생성하는 방법

생성자	설명
Random()	호출시 마다 다른 종자값(현재시간 이용)이 자동 설정된다.
Random(long seed)	매개값으로 주어진 종자값이 설정된다.

■ Random 클래스가 제공하는 메소드

리턴값	메소드(매개변수)	설명
boolean	nextBoolean()	boolean 타입의 난수를 리턴
double	nextDouble()	double 타입의 난수를 리턴($0.0 \leq \sim < 1.0$)
int	nextInt()	int 타입의 난수를 리턴($-2^{32} \leq \sim \leq 2^{32}-1$);
int	nextInt(int n)	int 타입의 난수를 리턴($0 \leq \sim < n$)



씨드(Seed) 기반의 난수 생성

```
public static void main(String[] args)
{
    Random rand=new Random(12);
    for(int i=0; i<100; i++)
        System.out.println(rand.nextInt(1000));
}
```

12라는 씨앗!이 전달되었으니, 12를 심어서
만들어지는 난수가 총 100개 생성된다.

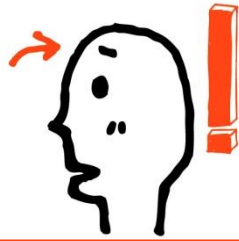
씨앗! 이 동일하면 생성되는 난수의 패턴은 100% 동일!
이렇듯 컴퓨터의 난수는 씨앗을 기반으로 생성되기 때문에 가짜 난수
(Pseudo-random number)라 불린다.

```
public static void main(String[] args)
{
    Random rand=new Random(12);
    rand.setSeed(System.currentTimeMillis());
    for(int i=0; i<100; i++)
        System.out.println(rand.nextInt(1000));
}
```

현재 시간을 밀리 초 단위로 반환

Random 클래스의 디폴트 생성자 내에서는
왼편의 코드와 같이 씨드를 설정한다.

매 실행 시마다 다른 유형의 난수를 발생시키는 방법



난수의 발생의 분포도 확인

```
import java.util.Random;

public class RandomExample
{
    private final int randomRange = 10;
    private int loopCount = 1000000;
    private int[] randomTable = new int[randomRange];
    private Random random = new Random(2);

    public static void main(String[] args)
    {
        RandomExample example = new RandomExample();
        example.init();
        example.generate();
        example.printStatus();
    }

    public void init()
    {
        for (int i = 0; i < randomTable.length; i++)
        {
            randomTable[i] = 0;
        }
    }
}
```



난수의 발생의 분포도 확인

```
public void generate()
{
    for (int i = 0; i < loopCount; i++)
    {
        int randomVlu = random.nextInt(randomRange);
        randomTable[randomVlu]++;
    }

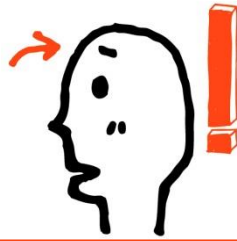
    public void printStatus()
    {
        for (int i = 0; i < randomTable.length; i++)
        {
            System.out.println("Random value (" + i + ") = " + randomTable[i]);
        }
    }
}
```

Random value (0) = 99628
Random value (1) = 100097
Random value (2) = 99847
Random value (3) = 99879
Random value (4) = 100111
Random value (5) = 100021
Random value (6) = 100075
Random value (7) = 100166
Random value (8) = 99891



난수의 생성

- 프로그램 사용자로부터 임의의 정수 A 와 z 를 입력받는다.
그리고 A 와 z 를 포함하여 그 사이에 있는 난수 10개를 생성하여 출력하는 프로그램을 작성해보자.



StringTokenizer 클래스 생성자와 주요메소드

❖ 문자열 분리 방법

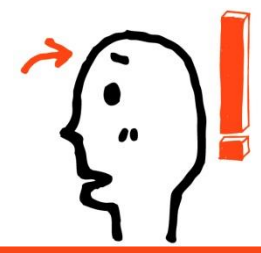
- String의 split() 메소드 이용
- java.util.StringTokenizer 클래스 이용

❖ String의 split()

- 정규표현식을 구분자로 해서 부분 문자열 분리
- 배열에 저장하고 리턴

홍길동&이수홍,박연수,김자바-최명호

```
String[] names = text.split("&|,-");
```



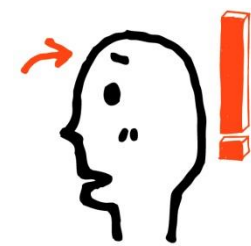
StringTokenizer 클래스 생성자와 주요메소드

□ 생성자

생성자	설명
<code>StringTokenizer(String str)</code>	<code>str</code> 스트링으로 파싱한 스트링 토크나이저 생성
<code>StringTokenizer(String str, String delim)</code>	<code>str</code> 스트링과 <code>delim</code> 구분 문자로 파싱한 스트링 토크나이저 생성
<code>StringTokenizer(String str, String delim, boolean returnDelims)</code>	<code>str</code> 스트링과 <code>delim</code> 구분 문자로 파싱한 스트링 토크나이저 생성. <code>returnDelims</code> 가 <code>true</code> 이면 <code>delim</code> 이 포함된 문자도 토크에 포함된다.

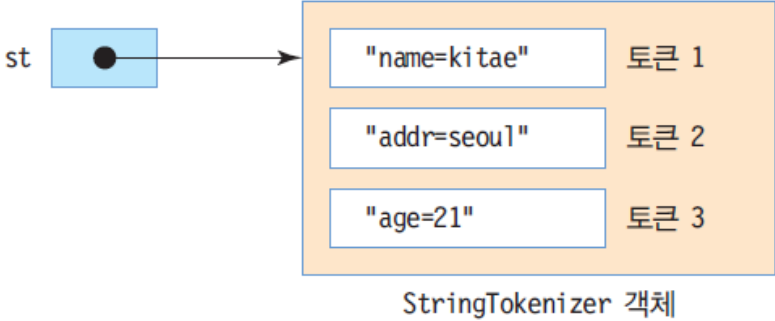
□ 주요 메소드

생성자	설명
<code>int countTokens()</code>	스트링 토크나이저에 포함된 토크의 개수 리턴
<code>boolean hasMoreTokens()</code>	스트링 토크나이저에 다음 토크가 있으면 <code>true</code> 리턴
<code>String nextToken()</code>	다음 토크 리턴

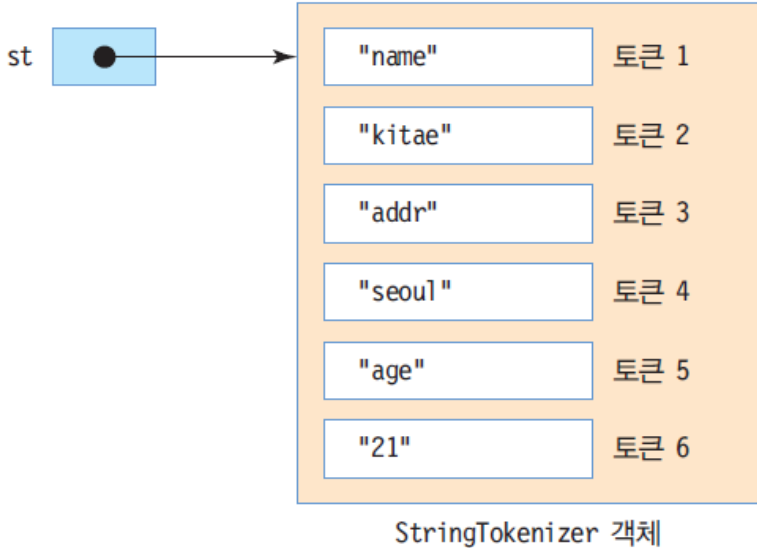


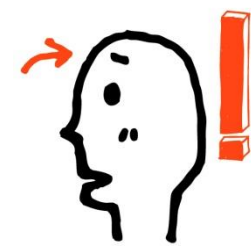
StringTokenizer 객체 생성과 문자열 분리

```
String query = "name=kitae&addr=seoul&age=21";  
StringTokenizer st = new StringTokenizer(query, "&");
```



```
StringTokenizer st = new StringTokenizer(query, "&=");
```





문자열 토큰(Token)의 구분

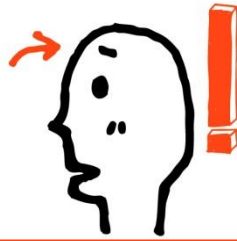
"PM:08:45"

이 문자열의 **구분자**가 :일 경우 **토큰**은 다음 세 가지

PM 08 45

위와 같이 토큰을 나누는 방법

```
StringTokenizer st = new StringTokenizer("PM:08:45", ":");  
public boolean hasMoreTokens()      반환할 토큰이 남아 있는가?  
public String nextToken()              다음 토큰을 반환
```



문자열 토큰(Token)의 구분

```
public static void main(String[] args) {  
    StringTokenizer st1 = new StringTokenizer("PM:08:45", ":");  
  
    while(st1.hasMoreTokens())  
        System.out.print(st1.nextToken() + ' ');  
    System.out.println();  
  
    StringTokenizer st2 = new StringTokenizer("12 + 36 - 8 / 2 = 44", "+-/= ");  
  
    while(st2.hasMoreTokens())  
        System.out.print(st2.nextToken() + ' ');  
    System.out.println();  
}
```

둘 이상의 구분자! 공백도 구분자에 포함!

명령 프롬프트

```
C:\JavaStudy>java TokenizeString  
PM 08 45  
12 36 8 2 44  
  
C:\JavaStudy>
```



Arrays 클래스의 배열 복사 메소드

```
public static int[] copyOf(int[] original, int newLength)
```

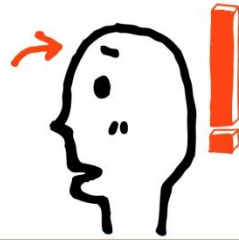
→ original에 전달된 배열을 첫 번째 요소부터 newLength의 길이만큼 복사

```
public static int[] copyOfRange(int[] original, int from, int to)
```

→ original에 전달된 배열을 인덱스 from부터 to 이전 요소까지 복사

```
public static void arraycopy(Object src, int srcPos, Object dest, int destPos, int length)
```

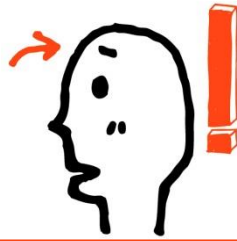
→ 배열 src의 srcPos에서 배열 dest의 destPos로 length 길이만큼 복사



copyOf 메소드 호출의 예

```
public static void main(String[] args) {  
    double[] arOrg = {1.1, 2.2, 3.3, 4.4, 5.5};  
  
    // 배열 전체를 복사  
    double[] arCpy1 = Arrays.copyOf(arOrg, arOrg.length);  
  
    // 세번째 요소까지만 복사  
    double[] arCpy2 = Arrays.copyOf(arOrg, 3);  
  
    for(double d : arCpy1)  
        System.out.print(d + "\t");  
    System.out.println();  
  
    for(double d : arCpy2)  
        System.out.print(d + "\t");  
    System.out.println();  
}
```

```
명령 프롬프트  
C:\JavaStudy>java CopyOfArrays  
1.1      2.2      3.3      4.4      5.5  
1.1      2.2      3.3  
C:\JavaStudy>
```

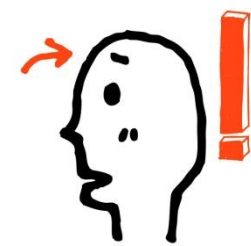


arraycopy 메소드 호출의 예

```
public static void main(String[] args) {  
    double[] org = {1.1, 2.2, 3.3, 4.4, 5.5};  
    double[] cpy = new double[3];  
  
    // 배열 org의 인덱스 1에서 배열 cpy 인덱스 0으로 세 개의 요소 복사  
    System.arraycopy(org, 1, cpy, 0, 3);  
  
    for(double d : cpy)  
        System.out.print(d + "\t");  
    System.out.println();  
}
```

C:\ 명령 프롬프트

```
C:\JavaStudy>java CopyOfSystem  
2.2      3.3      4.4  
  
C:\JavaStudy>_
```



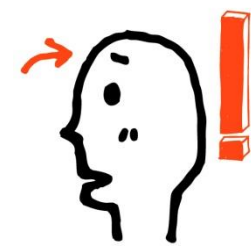
두 배열의 내용 비교

```
public static boolean equals(int[] a, int[] a2)
```

→ 매개변수 a와 a2로 전달된 배열의 내용을 비교하여 true 또는 false 반환

```
public static void main(String[] args) {  
    int[] ar1 = {1, 2, 3, 4, 5};  
    int[] ar2 = Arrays.copyOf(ar1, ar1.length);  
    System.out.println(Arrays.equals(ar1, ar2));  
}
```

```
C:\ 명령 프롬프트  
C:\JavaStudy>java ArrayEquals  
true  
C:\JavaStudy>
```



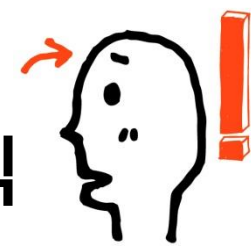
인스턴스 저장 배열의 비교 예

```
class INum {
    private int num;
    public INum(int num) {
        this.num = num;
    }
}

class ArrayObjEquals {
    public static void main(String[] args) {
        INum[] ar1 = new INum[3];
        INum[] ar2 = new INum[3];
        ar1[0] = new INum(1); ar2[0] = new INum(1);
        ar1[1] = new INum(2); ar2[1] = new INum(2);
        ar1[2] = new INum(3); ar2[2] = new INum(3);
        System.out.println(Arrays.equals(ar1, ar2));
    }
}
```

결과가 의미하는 바는? 어떤 식으로 비교?

```
명령 프롬프트
C:\JavaStudy>java ArrayObjEquals
false
C:\JavaStudy>
```

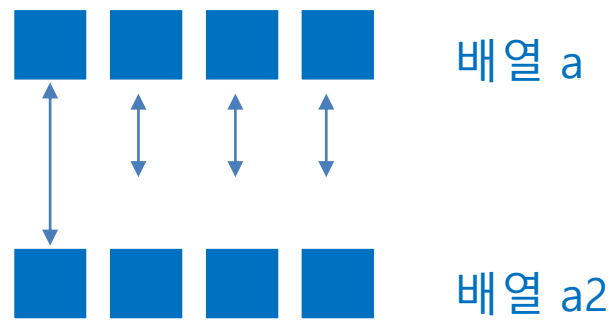


Arrays의 equals 메소드가 내용을 비교하는 방식

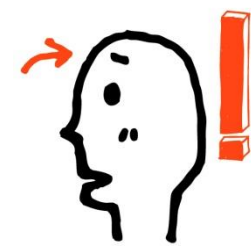
```
public static boolean equals(Object[] a, Object[] a2)
```

다음은 실제 Java의 Arrays.equals 메소드의 일부!

```
for (int i=0; i<length; i++) {  
    Object o1 = a[i];  
    Object o2 = a2[i];  
  
    if (!(o1==null ? o2==null : o1.equals(o2)))  
        return false;  
}
```



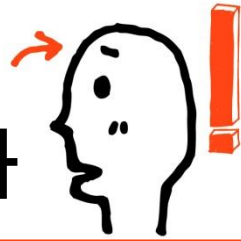
각 요소별로 Object 클래스의 equals 메소드로 비교



Object 클래스의 equals 메소드는?

```
public boolean equals(Object obj) {  
    if(this == obj) // 두 인스턴스가 동일 인스턴스이면  
        return true;  
    else  
        return false;  
} // 이렇듯 Object 클래스에 정의된 equals 메소드는 참조 값 비교를 한다.
```

따라서 Arrays 클래스의 equals 메소드가 두 배열의 내용 비교를 하도록 하려면
비교 대상의 equals 메소드를 내용 비교의 형태로 오버라이딩 해야 한다.



Object 클래스의 equals 메소드 오버라이딩 결과

```
class INum {  
    private int num;  
    public INum(int num) {  
        this.num = num;  
    }  
  
    @Override  
    public boolean equals(Object obj) {  
        if(this.num == ((INum)obj).num) }  
            return true;  
        else  
            return false;  
    }  
}
```

```
public static void main(String[] args) {  
    INum[] ar1 = new INum[3];  
    INum[] ar2 = new INum[3];  
    ar1[0] = new INum(1); ar2[0] = new INum(1);  
    ar1[1] = new INum(2); ar2[1] = new INum(2);  
    ar1[2] = new INum(3); ar2[2] = new INum(3);  
    System.out.println(Arrays.equals(ar1, ar2));  
}
```

```
C:\ 명령 프롬프트  
C:\JavaStudy>java ArrayObjEquals2  
true  
C:\JavaStudy>_
```



배열의 정렬: Arrays 클래스의 sort 메소드

```
public static void sort(int[] a)
```

→ 매개변수 a로 전달된 배열을 오름차순(Ascending Numerical Order)으로 정렬

```
public static void main(String[] args) {  
    int[] ar1 = {1, 5, 3, 2, 4};  
    double[] ar2 = {3.3, 2.2, 5.5, 1.1, 4.4};  
    Arrays.sort(ar1);  
    Arrays.sort(ar2);  
  
    for(int n : ar1)  
        System.out.print(n + "\t");  
    System.out.println();  
  
    for(double d : ar2)  
        System.out.print(d + "\t");  
    System.out.println();  
}
```

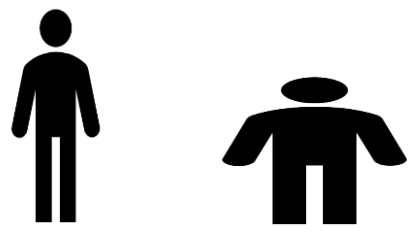




오름차순 정렬이란?

값이 작은 순에서 큰 순으로 세워 나가는 정렬
ex) 1, 2, 3, 4, 5, 6, 7

수의 경우는 오름차순에 대한 기준이 명확하다. 그렇다면 다음 두 사람을 오름차순으로 정렬해서 줄을 세운다고 하면? 한 사람은 키가 크고 다른 한 사람은 무게가 많이 나간다.



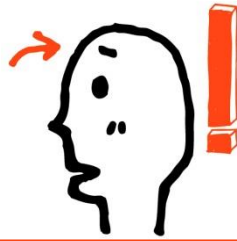
이 때에는 기준이 필요하다 오름차순 순서상 크고 작음에 대한 기준이 필요하다.
그래서 클래스를 정의할 때 오름차순 순서상 크고 작음에 대한 기준을 정의해야 한다.



compareTo 메소드 정의 기준

```
interface Comparable
    → int compareTo(Object o)
```

- 인자로 전달된 o가 작다면 양의 정수 반환
- 인자로 전달된 o가 크다면 음의 정수 반환
- 인자로 전달된 o와 같다면 0을 반환



클래스에 정의하는 오름차순 기준

```
class Person implements Comparable {
```

```
    private String name;
```

```
    private int age;
```

```
    . . . .
```

```
    @Override
```

```
    public int compareTo(Object o) {
```

```
        Person p = (Person)o;
```

```
        if(this.age > p.age)
```

```
            return 1;    // 인자로 전달된 o가 작다면 양의 정수 반환
```

```
        else if(this.age < p.age)
```

```
            return -1;   // 인자로 전달된 o가 크다면 음의 정수 반환
```

```
        else
```

```
            return 0;    // 인자로 전달된 o와 같다면 0을 반환
```

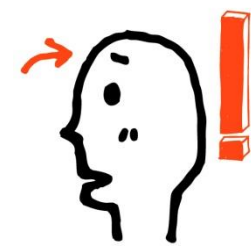
```
    }
```

```
    . . . .
```

```
}
```

Comparable 인터페이스를 구현한다는 것은
오름차순 순서상 크고 작음에 대한 기준을 제공한다는 의미

나이가 어릴수록 오름차순 순서상 작은 것으로 정의됨



다음과 같이 구현도 가능!

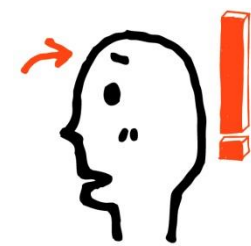
```
public int compareTo(Object o) {
    Person p = (Person)o;

    if(this.age > p.age)
        return 1;    // 인자로 전달된 o가 작다면 양의 정수 반환
    else if(this.age < p.age)
        return -1;    // 인자로 전달된 o가 크다면 음의 정수 반환
    else
        return 0;    // 인자로 전달된 o와 같다면 0을 반환
}
```



대신

```
public int compareTo(Object o) {
    Person p = (Person)o;
    return this.age - p.age;
}
```



배열에 저장된 인스턴스들의 정렬의 예

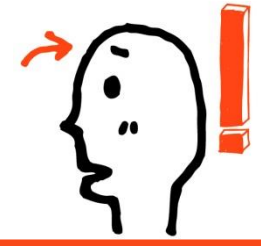
```
class Person implements Comparable {
    private String name;
    Private int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
    public int compareTo(Object o) {
        Person p = (Person)o;
        return this.age - p.age;
    }
    public String toString() {
        return name + ": " + age;
    }
}
```

```
public static void main(String[] args) {
    Person[] ar = new Person[3];
    ar[0] = new Person("Lee", 29);
    ar[1] = new Person("Goo", 15);
    ar[2] = new Person("Soo", 37);

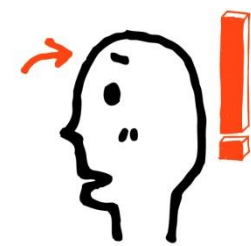
    Arrays.sort(ar);
    for(Person p : ar)
        System.out.println(p);
}
```

```
C:\ 명령 프롬프트
C:\JavaStudy>java ArrayObjSort
Goo: 15
Lee: 29
Soo: 37
C:\JavaStudy>_
```

정렬의 기준 수정하기

앞에 제시한 예제에서는 Person의 인스턴스들을 나이 순으로 정렬하였는데, 이를 이름의 길이 순으로 정렬이 되도록 수정해보자. 즉 이름의 길이가 짧은 인스턴스일수록 배열의 앞쪽에 위치하도록 예제를 수정해야 한다



배열의 탐색: 기본 자료형 값 대상

```
public static int binarySearch(int[] a, int key)
```

→ 배열 a에서 key를 찾아서 있으면 key의 인덱스 값, 없으면 0보다 작은 수 반환

binarySearch는 이진 탐색을 진행!

그리고 이진 탐색을 위해서는 탐색 이전에 데이터들이 오름차순으로 정렬되어 있어야 한다.

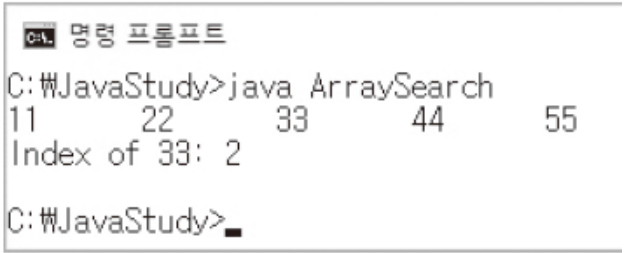


배열의 탐색: 기본 자료형 값 대상의 예

```
class ArraySearch {
    public static void main(String[] args) {
        int[] ar = {33, 55, 11, 44, 22};
        Arrays.sort(ar);    // 탐색 이전에 정렬이 선행되어야 한다.

        for(int n : ar)
            System.out.print(n + "\t");
        System.out.println();

        int idx = Arrays.binarySearch(ar, 33); // 배열 ar에서 33을 찾아라.
        System.out.println("Index of 33: " + idx);
    }
}
```





배열의 탐색: 인스턴스 대상

```
public static int binarySearch(Object[] a, Object key)
```

마찬가지로 탐색 대상들은 오름차순으로 정렬되어 있어야 한다.

그리고 탐색 대상의 확인 여부는 **compareTo** 메소드의 호출 결과를 근거로 한다.

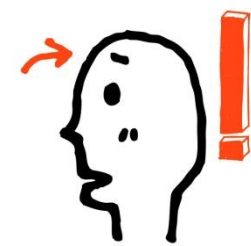
즉, 탐색 방식은 인스턴스의 내용 비교이다!



배열의 탐색: 인스턴스 대상의 예

```
class Person implements Comparable {
    private String name;
    private int age;
    . . .
    @Override
    public int compareTo(Object o) {
        Person p = (Person)o;
        return this.age - p.age;    // 나이가 같으면 0을 반환
    }
    . . .
    public static void main(String[] args) {
        Person[] ar = new Person[3];
        ar[0] = new Person("Lee", 29);
        ar[1] = new Person("Goo", 15);
        ar[2] = new Person("Soo", 37);
        Arrays.sort(ar);    // 탐색에 앞서 정렬을 진행
        int idx = Arrays.binarySearch(ar, new Person("Who are you?", 37));
        System.out.println(ar[idx]);
    }
}
```





탐색의 기준 변경

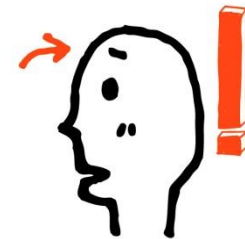
앞에 제시한 예제에서 탐색의 기준은 나이였다.
그런데 이 탐색의 기준이 이름이 되도록 예제를 수정해보자.

THANK YOU

실무에서 알아야 할 기술은 따로 있다! 자바를 다루는 기술



Android Java



01 제네릭 클래스의 이해와 설계

02 제네릭을 구성하는 다양한 문법적 요소



AppleBox와 OrangeBox 클래스의 설계

```
class AppleBox
{
    Apple item;
    public void store(Apple item) { this.item=item; }
    public Apple pullOut() { return item; }
}

class OrangeBox
{
    Orange item;
    public void store(Orange item) { this.item=item; }
    public Orange pullOut() { return item; }
}
```

Apple 상자와 Orange 상자를
구분한 모델

```
Class FruitBox
{
    Object item;
    public void store(Object item) { this.item=item; }
    public Object pullOut() { return item; }
}
```

무엇이든 담을 수 있는 상자 클래스

구현의 편의만 놓고 보면, FruitBox 클래스가 더 좋아 보인다. 하지만 FruitBox 클래스는 자료형에 안전하지 못하다는 단점이 있다.

물론 AppleBox와 OrangeBox는 구현의 불편함이 따르는 단점이 있다. 그러나 자료형에 안전하다는 장점이 있다.

AppleBox, OrangeBox의 장점인 자료형의 안전성과 FruitBox의 장점인 구현의 편의성을 한데 모은것이 제네릭이다!



자료형의 안전성에 대한 논의

```
public static void main(String[] args)
{
    FruitBox fBox1=new FruitBox();
    fBox1.store(new Orange(10));
    Orange org1=(Orange)fBox1.pullOut();
    org1.showSugarContent();

    FruitBox fBox2=new FruitBox();
    fBox2.store("오렌지");
    Orange org2=(Orange)fBox2.pullOut();
    org2.showSugarContent();
}
```

예외 발생지점

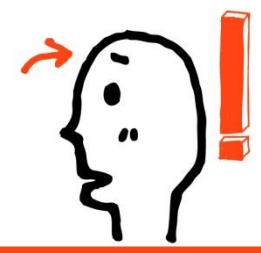
실행중간에 Class Casting Exception이 발생한다!
그런데 실행중간에 발생하는 예외는 컴파일 과정에서 발견되는 오류상황보다 발견 및 정정이 어렵다!
즉, 이는 자료형에 안전한 형태가 아니다.

```
public static void main(String[] args)
{
    OrangeBox fBox1=new OrangeBox();
    fBox1.store(new Orange(10));
    Orange org1=fBox1.pullOut();
    org1.showSugarContent();

    OrangeBox fBox2=new OrangeBox();
    fBox2.store("오렌지");
    Orange org2=fBox2.pullOut();
    org2.showSugarContent();
}
```

에러 발생지점

컴파일 과정에서, 메소드 store에 문자열 인스턴스가 전달될 수 없음이 발견된다. 이렇듯 컴파일 과정에서 발견된 자료형 관련 문제는 발견 및 정정이 간단하다.
즉, 이는 자료형에 안전한 형태이다.



제네릭(Generics) 클래스 설계

```
class FruitBox
{
    Object item;
    public void store(Object item){this.item=item;}
    public Object pullOut(){return item;}
}
```

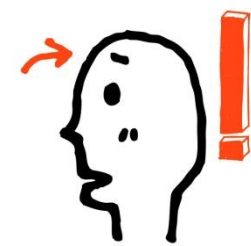
↓ 제네릭 정의의 결과

T라는 이름이 매개변수화 된 자료형임을 나타냄

```
class FruitBox <T>
{
    T item;
    public void store(T item ) {this.item=item;}
    public T pullOut() {return item;}
}
```

T에 해당하는 자료형의 이름은 인스턴스를 생성하는 순간에 결정이 된다!

제네릭 클래스가 자료형의 안전과 정의의 편의라는 두 마리 토끼를 실제로 잡았는지 생각해 보자!



제네릭 클래스 기반 인스턴스 생성

```
FruitBox<Orange> orBox=new FruitBox<Orange>();
```

T를 Orange로 결정해서 FruitBox의 인스턴스를 생성하고 이를 참조할 수 있는 참조변수를 선언해서 참조 값을 저장한다!

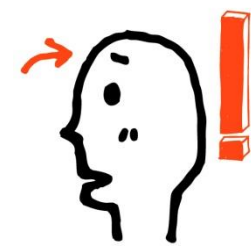
```
FruitBox<Apple> apBox=new FruitBox<Apple>();
```

T를 Apple로 결정해서 FruitBox의 인스턴스를 생성하고 이를 참조할 수 있는 참조변수를 선언해서 참조 값을 저장한다!

```
public static void main(String[] args)
{
    FruitBox<Orange> orBox=new FruitBox<Orange>();
    orBox.store(new Orange(10));
    Orange org=orBox.pullOut();
    org.showSugarContent();

    FruitBox<Apple> apBox=new FruitBox<Apple>();
    apBox.store(new Apple(20));
    Apple app=apBox.pullOut();
    app.showAppleWeight();
}
```

인스턴스 생성시 결정된 T의 자료형에
일치하지 않으면 컴파일 에러가 발생
하므로 자료형에 안전한 구조임!



제네릭 클래스 기반 인스턴스 생성

❖ 제네릭 타입 사용 여부에 따른 비교

- 제네릭 타입 사용한 경우
 - 클래스 선언할 때 타입 파라미터 사용
 - 컴파일 시 타입 파라미터가 구체적인 클래스로 변경

```
public class Box<T> {  
    private T t;  
    public T get() { return t; }  
    public void set(T t) { this.t = t; }  
}
```

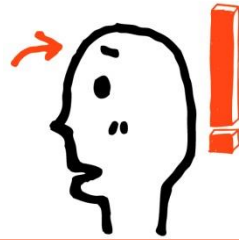
```
public class Box<String> {  
    private String t;  
    public void set(String t) { this.t = t; }  
    public String get() { return t; }  
}
```

```
Box<String> box = new Box<String>();  
box.set("hello");  
String str = box.get();
```

```
Box<Integer> box = new Box<Integer>();
```

```
public class Box<Integer> {  
    private Integer t;  
    public void set(Integer t) { this.t = t; }  
    public Integer get() { return t; }  
}
```

```
Box<Integer> box = new Box<Integer>();  
box.set(6);  
int value = box.get();
```



제네릭 클래스 기반 인스턴스 생성

❖ 제네릭 타입은 두 개 이상의 타입 파라미터 사용 가능

■ 각 타입 파라미터는 콤마로 구분

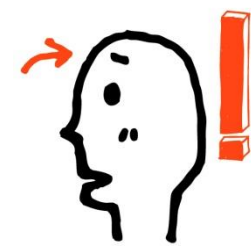
- **Ex) class**<K, V, ...> { ... }
- **interface**<K, V, ...> { ... }

```
public class Product<T, M> {  
    private T kind;  
    private M model;  
  
    public T getKind() { return this.kind; }  
    public M getModel() { return this.model; }  
  
    public void setKind(T kind) { this.kind = kind; }  
    public void setModel(M model) { this.model = model; }  
}
```

```
Product<Tv, String> product = new Product<Tv, String>();
```

■ 자바 7부터는 다이아몬드 연산자 사용해 간단히 작성과 사용 가능

```
Product<Tv, String> product = new Product<>();
```

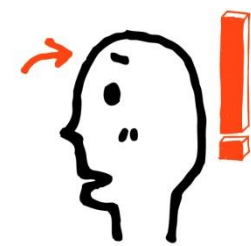
제네릭 클래스 기반 인스턴스 생성

```
class DBox<L, R> {
    private L left;    // 왼쪽 수납 공간
    private R right;   // 오른쪽 수납 공간

    public void set(L o, R r) {
        left = o;
        right = r;
    }

    @Override
    public String toString() {
        return left + " & " + right;
    }
}

public static void main(String[] args) {
    DBox<String, Integer> box = new DBox<String, Integer>();
    box.set("Apple", 25);
    System.out.println(box);
}
```



제네릭 메소드의 정의와 호출

❖ 제네릭 메소드

- 매개변수 타입과 리턴 타입으로 타입 파라미터를 갖는 메소드
- 제네릭 메소드 선언 방법
 - 리턴 타입 앞에 “<>” 기호를 추가하고 타입 파라미터 기술
 - 타입 파라미터를 리턴 타입과 매개변수에 사용

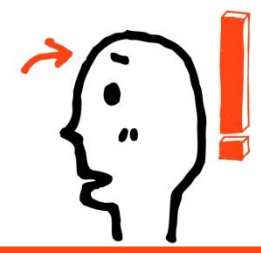
```
public <타입파라미터,...> 리턴타입 메소드명(매개변수,...) { ... }
```

```
public <T> Box<T> boxing(T t) { ... }
```

■ 제네릭 메소드 호출하는 두 가지 방법

```
리턴타입 변수 = <구체적타입> 메소드명(매개값);           //명시적으로 구체적 타입 지정
리턴타입 변수 = 메소드명(매개값);                       //매개값을 보고 구체적 타입을 추정

Box<Integer> box = <Integer>boxing(100);                 //타입 파라미터를 명시적으로 Integer 로 지정
Box<Integer> box = boxing(100);                          //타입 파라미터를 Integer 으로 추정
```

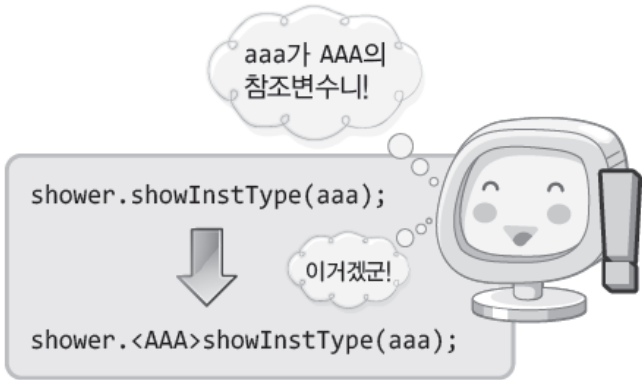


제네릭 메소드의 정의와 호출

```
class InstanceTypeShower
{
    int showCnt=0;

    public <T> void showInstType(T inst)
    {
        System.out.println(inst);
        showCnt++;
    }

    void showPrintCnt() { . . . . . }
}
```



클래스의 메소드만 부분적으로 제네릭화 할 수 있다!

```
public static void main(String[] args)
{
    AAA aaa=new AAA();
    BBB bbb=new BBB();

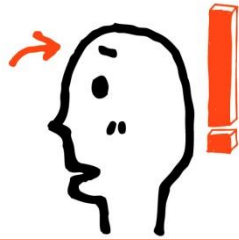
    InstanceTypeShower shower=new InstanceTypeShower();
    shower.<AAA>showInstType(aaa);
    shower.<BBB>showInstType(bbb);
    shower.showPrintCnt();
}
```

제네릭 메소드의 호출과정에서 전달되는 인자를 통해서 제네릭 자료형을 결정할 수 있으므로 자료형의 표현은 생략 가능하다!

일반적인 메소드 호출방법!

```
shower.showInstType(aaa);
shower.showInstType(bbb);
```

제네릭 메소드의 호출방법! T를 AAA로, BBB로 결정하여 호출하고 있다!



제네릭 메소드의 정의와 호출

클래스 전부가 아닌 메소드 하나에 대해 제네릭으로 정의

```
class BoxFactory {  
    public static <T> Box<T> makeBox(T o) {  
        Box<T> box = new Box<T>();    // 상자를 생성하고,  
        box.set(o);    // 전달된 인스턴스를 상자에 담아서,  
        return box;    // 상자를 반환한다.  
    }  
}
```

제네릭 메소드의 T는 메소드 호출 시점에 결정한다.

```
Box<String> sBox = BoxFactory.<String>makeBox("Sweet");  
Box<Double> dBox = BoxFactory.<Double>makeBox(7.59);    // 7.59에 대해 오토 박싱 진행됨
```

다음과 같이 타입 인자 생략 가능

```
Box<String> sBox = BoxFactory.makeBox("Sweet");  
Box<Double> dBox = BoxFactory.makeBox(7.59);    // 7.59에 대해 오토 박싱 진행됨
```



제네릭 메소드와 둘 이상의 자료형

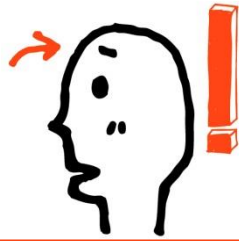
```
class InstanceTypeShower2
{
    public <T, U> void showInstType(T inst1, U inst2)
    {
        System.out.println(inst1);
        System.out.println(inst2);
    }
}
```

T, U와 같은 문자는 상징적이다.
따라서 타 문자로도 대체 가능!

```
public static void main(String[] args)
{
    AAA aaa=new AAA();
    BBB bbb=new BBB();

    InstanceTypeShower2 shower=new InstanceTypeShower2();
    shower.<AAA, BBB>showInstType(aaa, bbb);
    shower.showInstType(aaa, bbb);
}
```

마찬가지로 메소드 호출 시,
자료형의 정보는 생략이 가능하다!

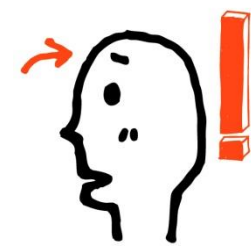


‘매개변수화 타입’을 ‘타입 인자’로 전달

```
class Box<T> {  
    private T ob;  
  
    public void set(T o) {  
        ob = o;  
    }  
    public T get() {  
        return ob;  
    }  
}
```

```
public static void main(String[] args) {  
    Box<String> sBox = new Box<>();  
    sBox.set("I am so happy.");  
  
    Box<Box<String>> wBox = new Box<>();  
    wBox.set(sBox);  
  
    Box<Box<Box<String>>> zBox = new Box<>();  
    zBox.set(wBox);  
  
    System.out.println(zBox.get().get().get());  
}
```

```
CA. 명령 프롬프트  
C:\JavaStudy>java BoxInBox  
I am so happy.  
C:\JavaStudy>_
```



매개변수의 자료형 제한

```
public static <T extends AAA> void myMethod(T param) { . . . . . }
```

T가 AAA를 상속(AAA가 클래스인 경우) 또는 구현(AAA가 인터페이스인 경우)하는 클래스의 자료형이 되어야 함을 명시함

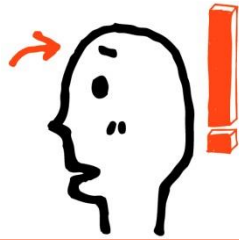
```
public static <T extends SimpleInterface> void showInstanceAncestor(T param)
{
    param.showYourName();
}
```

인터페이스 이름
인자는 SimpleInterface를 구현하는 클래스
의 인스턴스이어야 함!

```
public static <T extends UpperClass> void showInstanceName(T param)
{
    param.showYourAncestor();
}
```

클래스 이름
UpperClass를 상속하는 클래스의 인스턴스이어야 함!

키워드 extends는 매개변수의 자료형을 제한하는 용도로도 사용된다!



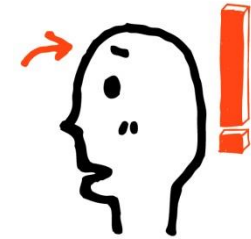
매개변수의 자료형 제한

```
class Box<T extends Number> {...}
```

→ 인스턴스 생성 시 타입 인자로 Number 또는 이를 상속하는 클래스만 올 수 있음

```
class Box<T extends Number> {  
    private T ob;  
  
    public void set(T o) {  
        ob = o;  
    }  
    public T get() {  
        return ob;  
    }  
}
```

```
public static void main(String[] args) {  
    Box<Integer> iBox = new Box<>();  
    iBox.set(24);  
  
    Box<Double> dBox = new Box<>();  
    dBox.set(5.97);  
    . . .  
}
```

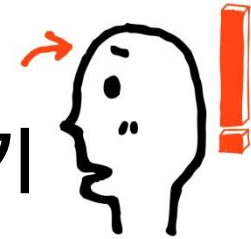



타입 인자 제한의 효과

```
class Box<T> {
    private T ob;
    ....
    public int toIntValue() {
        return ob.intValue();    // ERROR!
    }
}
```

```
class Box<T extends Number> {
    private T ob;
    ....
    public int toIntValue() {
        return ob.intValue();    // OK!
    }
}
```

이 효과가 타입 인자를 제한하는 실질적인 이유인 경우가 많다.



제네릭 클래스의 타입 인자를 인터페이스로 제한하기

```
interface Eatable { public String eat(); }

class Apple implements Eatable {
    public String eat() {
        return "It tastes so good!";
    }
    . . . .
}

class Box<T extends Eatable> {
    T ob;

    public void set(T o) { ob = o; }
    public T get() {
        System.out.println(ob.eat());    // Eatable로 제한하였기에 eat 호출 가능
        return ob;
    }
}
```



하나의 클래스와 하나의 인터페이스에 대해 동시 제한

```
class Box<T extends Number & Eatable> {...}
```

Number는 클래스 이름 Eatable은 인터페이스 이름



제네릭 메소드와 배열

제네릭 메소드로의 배열 전달

배열도 인스턴스이므로 제네릭 매개변수에 전달이 가능하다. 하지만 이렇게 전달을 하면 다음과 같은 문장을 쓸 수 없다!

```
System.out.println(arr[i]);
```

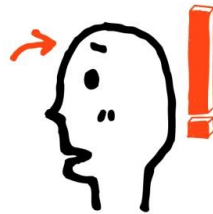
다음과 같이 매개변수를 선언하면, 매개변수에 전달되는 참조 값을 배열 인스턴스의 참조 값으로 제한할 수 있다.

```
T[] arr
```

그리고 이렇게 되면 참조 값은 배열 인스턴스의 참조 값임이 100% 보장 되므로 [] 연산을 허용한다.

```
public static <T> void showArrayData(T[] arr)
{
    for(int i=0; i<arr.length; i++)
        System.out.println(arr[i]);
}
```

이렇듯 [] 연산이 필요하다면 매개변수의 선언을 통해서 전달 되는 참조 값을 배열의 참조 값으로 제한해야 한다.



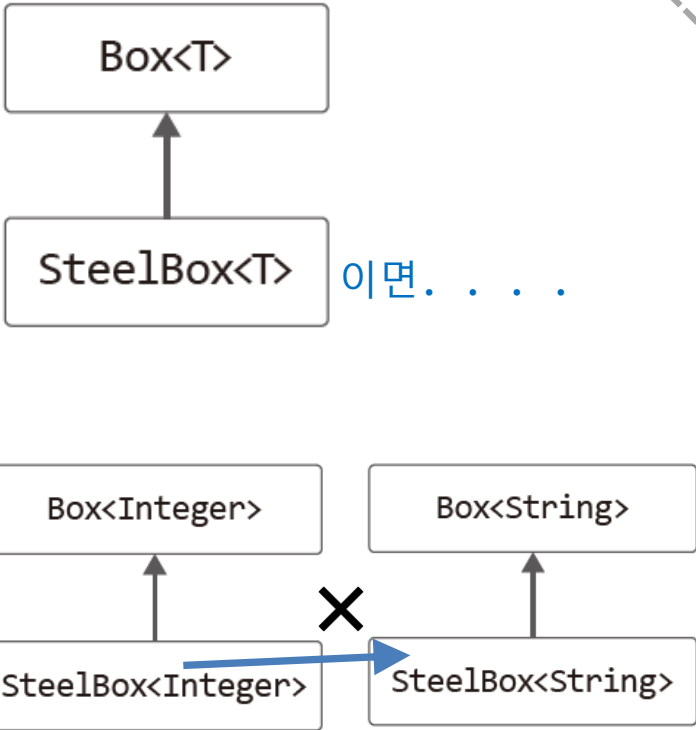
제네릭 변수의 참조와 상속의 관계

```
class Box<T> {
    protected T ob;
    public void set(T o) { ob = o; }
    public T get() { return ob; }
}

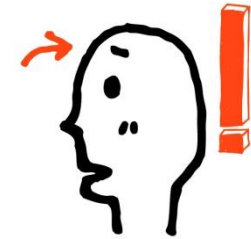
class SteelBox<T> extends Box<T> {
    public SteelBox(T o) { // 제네릭 클래스의 생성자
        ob = o;
    }
}

Box<Integer> iBox = new SteelBox<>(7959);
↔ Box<Integer> iBox = new SteelBox<Integer>(7959);

Box<String> sBox = new SteelBox<>("Simple");
↔ Box<String> sBox = new SteelBox<String>("Simple");
```



이다. !!

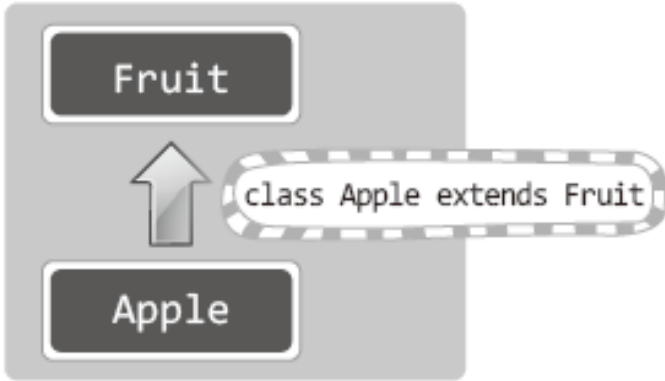


제네릭 변수의 참조와 상속의 관계

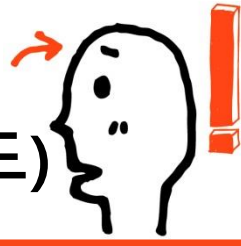
```
public void ohMethod (FruitBox<Fruit> param) { . . . }
```

ohMehod의 인자로 전달될 수 있는 참조 값의 자료형은

- ➔ FruitBox<Fruit>의 인스턴스 참조 값
- ➔ FruitBox<Fruit>을 상속하는 인스턴스의 참조 값



왼쪽과 같은 상속 구조에서
FruitBox<Apple> 클래스는 ohMethod의
인자가 될수 있을까? NO!
반드시 키워드 extends를 이용해서 상속이
명시된 대상만 인자로 전달될 수 있다.
... extends FruitBox<Fruit>



와일드카드의 설명에 앞서(제네릭 메소드 vs. 일반 메소드)

Box<Integer>의 인스턴스, Box<String>의 인스턴스를 인자로 전달 가능

```
public static <T> void peekBox(Box<T> box) {  
    System.out.println(box);  
}
```

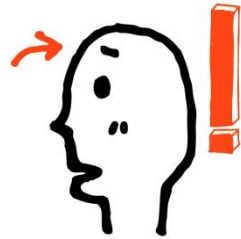
Box<Integer>의 인스턴스, Box<String>의 인스턴스를 인자로 전달 가능할 것 같지만 불가능

```
public static void peekBox(Box<Object> box) {  
    System.out.println(box);  
}
```

"Box<Object>와 Box<String>은 상속 관계를 형성하지 않는다."

"Box<Object>와 Box<Integer>은 상속 관계를 형성하지 않는다."

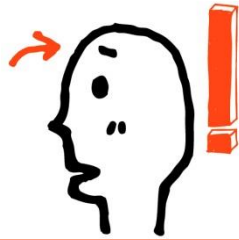
그러나 와일드카드를 사용하면 일반 메소드도 이 두 인스턴스를 인자로 받을 수 있다.



와일드카드

Box<Integer>의 인스턴스, Box<String>의 인스턴스를 인자로 전달 가능

```
public static void peekBox(Box<?> box) {  
    System.out.println(box);  
}
```

와일드카드와 제네릭 변수의 선언

<? extends Fruit> _____

<? extends Fruit>가 의미하는 바는 “Fruit을 상속하는 모든 클래스”이다

➔ `FruitBox<? extends Fruit> box1 = new FruitBox<Fruit>();`

➔ `FruitBox<? extends Fruit> box2 = new FruitBox<Apple>();`

`FruitBox<Fruit>` 인스턴스의 참조 값도,

`FruitBox<Apple>` 인스턴스의 참조 값도

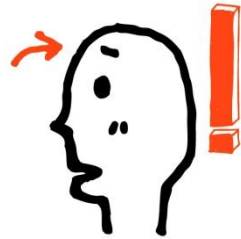
인자로 전달받을 수 있는 매개변수의 선언에는 와일드카드 문자 ?가 사용된다!

`FruitBox<?> box;` _____

자료형에 상관없이 `FruitBox<T>`의 인스턴를 참조에 사용되는 참조변수,

다음 선언과 동일하다.

➔ `FruitBox<? extends Object> box;`

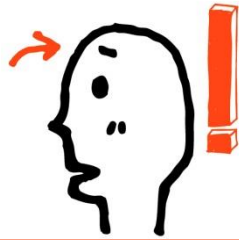


기능적으로는 두 메소드 완전 동일

```
public static <T> void peekBox(Box<T> box) {  
    System.out.println(box);  
} // 제네릭 메소드의 정의
```

```
public static void peekBox(Box<?> box) {  
    System.out.println(box);  
} // 와일드카드 기반 메소드 정의
```

그러나 와일드카드 기반 메소드 정의가 더 간결하므로 우선시 해야 한다고 권고하고 있다.
메소드의 정의가 복잡해질수록 와일드카드 기반 메소드 정의가 더 간결해 보인다.



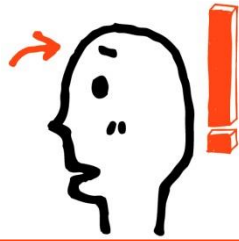
하위 클래스를 제한하는 용도의 와일드 카드

```
FruitBox<? extends Apple> boundedBox;
```

- ~을 상속하는 클래스라면 무엇이든지
- Apple을 상속하는 클래스의 인스턴스라면 무엇이든지 참조 가능한 참조변수 선언

```
FruitBox<? super Apple> boundedBox;
```

- ~이 상속하는 클래스라면 무엇이든지
- Apple이 상속하는 클래스의 인스턴스라면 무엇이든지 참조 가능한 참조변수 선언



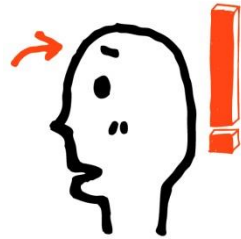
상한 제한된 와일드카드(Upper-Bounded Wildcards)

```
public static void peekBox(Box<?> box) {  
    System.out.println(box);  
}
```

```
public static void peekBox(Box<? extends Number> box) {  
    System.out.println(box);  
}
```

box는 Box<T> 인스턴스의 참조 값을 전달받는 매개변수이다.

→ 단 전달되는 인스턴스의 T는 Number 또는 이를 상속하는 하위 클래스이어야 함

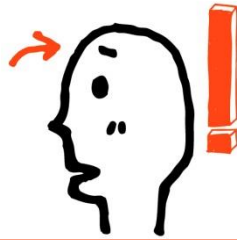


하한 제한된 와일드카드(Lower-Bounded Wildcards)

```
public static void peekBox(Box<? super Integer> box) {  
    System.out.println(box);  
}
```

box는 Box<T> 인스턴스의 참조 값을 전달받는 매개변수이다.

- 단 전달되는 인스턴스의 T는 Integer 또는 Integer가 상속하는 클래스이어야 함
- 즉 위 메소드의 인자로 전달 가능한 인스턴스는 Box<Integer>, Box<Number>, Box<Object>으로 제한됨



제네릭 클래스의 다양한 상속방법

```
class AAA<T>
{
    T itemAAA;
}
class BBB<T> extends AAA<T>
{
    T itemBBB;
}
```

왼쪽과 같이 제네릭 클래스도 상속이 가능하다. 그리고 이 경우 다음과 같이 인스턴스를 생성하게 된다.

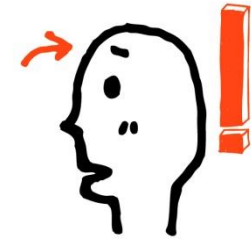
```
BBB<String> myString=new BBB<String>( );
BBB<Integer> myInteger=new BBB<Integer>( );
```

이 경우 T는 각각 String과 Integer로 대체되어 인스턴스가 생성된다.

```
class AAA<T>
{
    T itemAAA;
}
class BBB extends AAA<String>
{
    int itemBBB;
}
```

왼쪽과 같이 제네릭 클래스의 자료형을 결정해서 상속하는 것도 가능하다.

이 경우, BBB 클래스는 제네릭과 관련 없는 클래스가 되어, 일반적인 방법으로 인스턴스를 생성하면 된다.



기본자료형의 이름은 제네릭에 사용 불가!

컴파일 불가능한 문장들!

```
FruitBox<int> fb1=new FruitBox<int>();
```

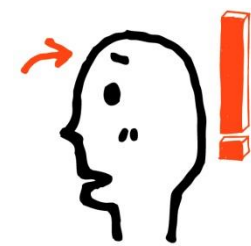
```
FruitBox<double> fb1=new FruitBox<double>();
```

기본 자료형 정보를 이용해서는 제네릭 클래스의 인스턴스 생성이 불가능하다!

제네릭은 클래스와 인스턴스에 관한 이야기이다!

그럼 기본 자료형 정보를 대상으로는 제네릭 인스턴스의 생성이 필요한 상황에서는 어떻게 하죠?

위 질문에 Wrapper 클래스를 떠올리기 바란다! 이와 관련된 이야기는 이후에 계속된다!



SelfCheck

1. 다음 코드가 실행되도록 swapBox 메소드를 정의하되, Box<T> 인스턴스를 인자로 전달받을 수 있도록 정의하자. 단 이때 Box<T> 인스턴스의 T는 Number 또는 이를 상속하는 하위 클래스만 올 수 있도록 제한된 매개변수를 선언을 하자

```
class BoxSwapDemo {
    // 이 위치에 swapBox 메소드 정의하자.
    public static void main(String[] args) {
        Box<Integer> box1 = new Box<>();
        box1.set(99);

        Box<Integer> box2 = new Box<>();
        box2.set(55);

        System.out.println(box1.get() + " & " + box2.get());
        swapBox(box1, box2);
        System.out.println(box1.get() + " & " + box2.get());
    }
}

class Box<T> {
    private T ob;

    public void set(T o) {
        ob = o;
    }

    public T get() {
        return ob;
    }
}
```

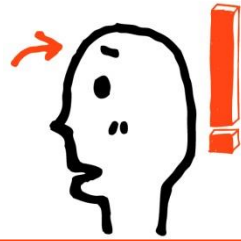
결과 :
99 & 55
55 & 99



Android Java

21장

컬렉션 프레임워크



컬렉션 프레임워크

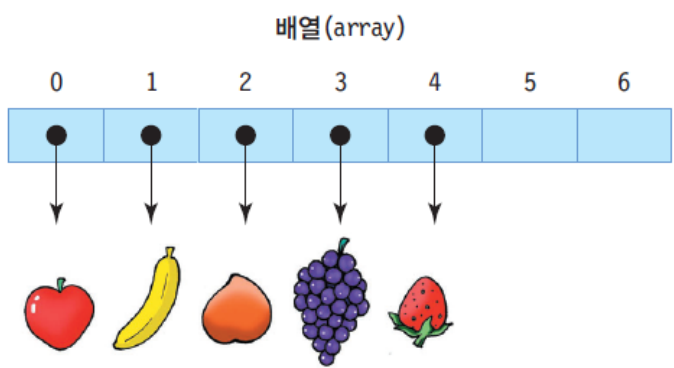
- 01** 컬렉션 프레임워크의 이해
- 02** `Collection<E>` 인터페이스를 구현하는 제네릭 클래스들
- 03** `Set<e>` 인터페이스를 구현하는 컬렉션 클래스들
- 04** `Map(K, V)` 인터페이스를 구현하는 컬렉션 클래스들



자바에는 컬렉션이라는 편리한 것이 있다1

□ 컬렉션

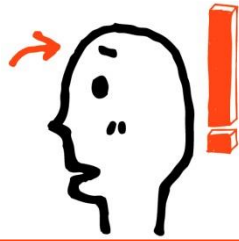
- 요소(element)라고 불리는 가변 개수의 객체들의 저장소
 - 객체들의 컨테이너라고도 불림
 - 요소의 개수에 따라 크기 자동 조절
 - 요소의 삽입, 삭제에 따른 요소의 위치 자동 이동
- 고정 크기의 배열을 다루는 어려움 해소
- 다양한 객체들의 삽입, 삭제, 검색 등의 관리 용이



- 고정 크기 이상의 객체를 관리할 수 없다.
- 배열의 중간에 객체가 삭제되면 응용프로그램에서 자리를 옮겨야 한다.



- 가변 크기로서 객체의 개수를 염려할 필요 없다.
- 컬렉션 내의 한 객체가 삭제되면 컬렉션이 자동으로 자리를 옮겨준다.



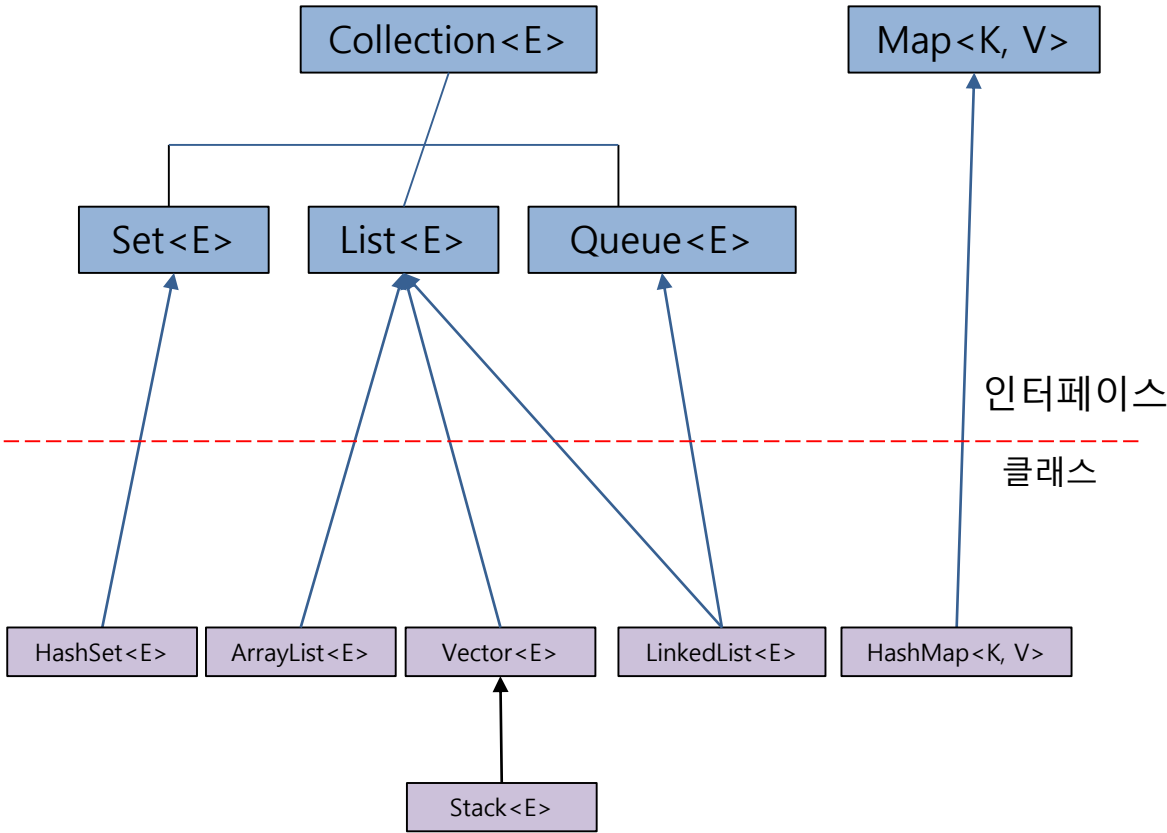
자바에는 컬렉션이라는 편리한 것이 있다1

- ▶ **컬렉션(Collection)** : 객체들을 제어/관리하기 위한 클래스를 의미
- ▶ **컬렉션 객체(Collection Object)** : 여러 개의 요소(element)를 묶어서 하나의 객체로 만든 것
- ▶ 컬렉션은 배열과 같이 멤버 객체를 관리하기 위한 클래스들을 통틀어 이르는 말
- ▶ 컬렉션은 데이터를 묶어서 관리 및 처리를 쉽게 하기 위해 많이 사용되는 기능 중에 하나
- ▶ 컬렉션을 사용하는 목적은 배열과 같지만 제공하는 기능이 배열보다 좀더 많음
- ▶ **컬렉션 프레임워크(framework)** : 컬렉션 클래스들과 인터페이스들의 집합
- ▶ 컬렉션 프레임워크는 자료구조 및 알고리즘을 구현해 놓은 일종의 라이브러리!
제네릭 기반으로 구현이 되어 있다.

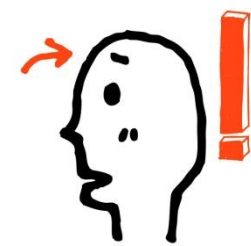


컬렉션 프레임워크의 기본골격

❖ 컬렉션 프레임워크의 인터페이스 구조

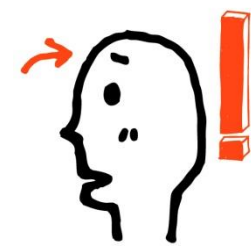


- ✓ **Collection<E>** 인터페이스를 구현하는 제네릭 클래스
 - ➔ 인스턴스 단위의 데이터 저장 기능 제공
 - (배열과 같이 단순 인스턴스 참조 값 저장)
- ✓ **Map<K, V>**
 - ➔ key-value 구조의 인스턴스 저장 기능 제공



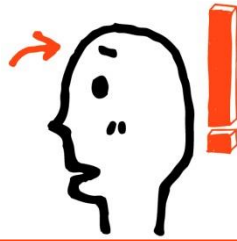
컬렉션과 제네릭

- 컬렉션은 제네릭(generics) 기법으로 구현됨
- 컬렉션의 요소는 객체만 가능
 - ▣ 기본적으로 int, char, double 등의 기본 타입 사용 불가
 - JDK 1.5부터 자동 박싱/언박싱 기능으로 기본 타입 사용 가능
- 제네릭
 - ▣ 특정 타입만 다루지 않고, 여러 종류의 타입으로 변신할 수 있도록 클래스나 메소드를 일반화시키는 기법
 - <E>, <K>, <V> : 타입 매개 변수
 - 요소 타입을 일반화한 타입
 - ▣ 제네릭 클래스 사례
 - 제네릭 벡터 : Vector<E>
 - E에 특정 타입으로 구체화
 - 정수만 다루는 벡터 Vector<Integer>
 - 문자열만 다루는 벡터 Vector<String>



Vector<E>

- Vector<E>의 특성
 - ▣ java.util.Vector
 - <E>에서 E 대신 요소로 사용할 특정 타입으로 구체화
 - ▣ 여러 객체들을 삽입, 삭제, 검색하는 컨테이너 클래스
 - 배열의 길이 제한 극복
 - 원소의 개수가 넘쳐나면 자동으로 길이 조절
 - ▣ Vector에 삽입 가능한 것
 - 객체, null
 - 기본 타입은 박싱/언박싱으로 Wrapper 객체로 만들어 저장
 - ▣ Vector에 객체 삽입
 - 벡터의 맨 뒤에 객체 추가
 - 벡터 중간에 객체 삽입
 - ▣ Vector에서 객체 삭제
 - 임의의 위치에 있는 객체 삭제 가능 : 객체 삭제 후 자동 자리 이동



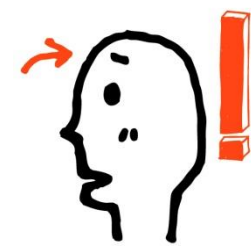
Vector<E>

▶ List 인터페이스를 구현하는 ArrayList와 Vector 클래스

▷ Vector 객체를 생성하는 방법

```
사용법 : Vector<[데이터형]> [변수 이름] = new Vector <[데이터형]> ( );  
        Vector<[데이터형]> [변수 이름] = new Vector <[데이터형]> ( Collection c );  
        Vector<[데이터형]> [변수 이름] = new Vector <[데이터형]> ( int 초깃값 );  
        Vector<[데이터형]> [변수 이름] = new Vector <[데이터형]> ( int 초깃값, int 증가값 );  
사용예 : Vector<String> strVector = new Vector<String>();  
        Vector<String> strVector = new Vector<String>(stringVector);  
        Vector<String> strVector = new Vector<String>(10);  
        Vector<String> strVector = new Vector<String>(10 , 5);
```

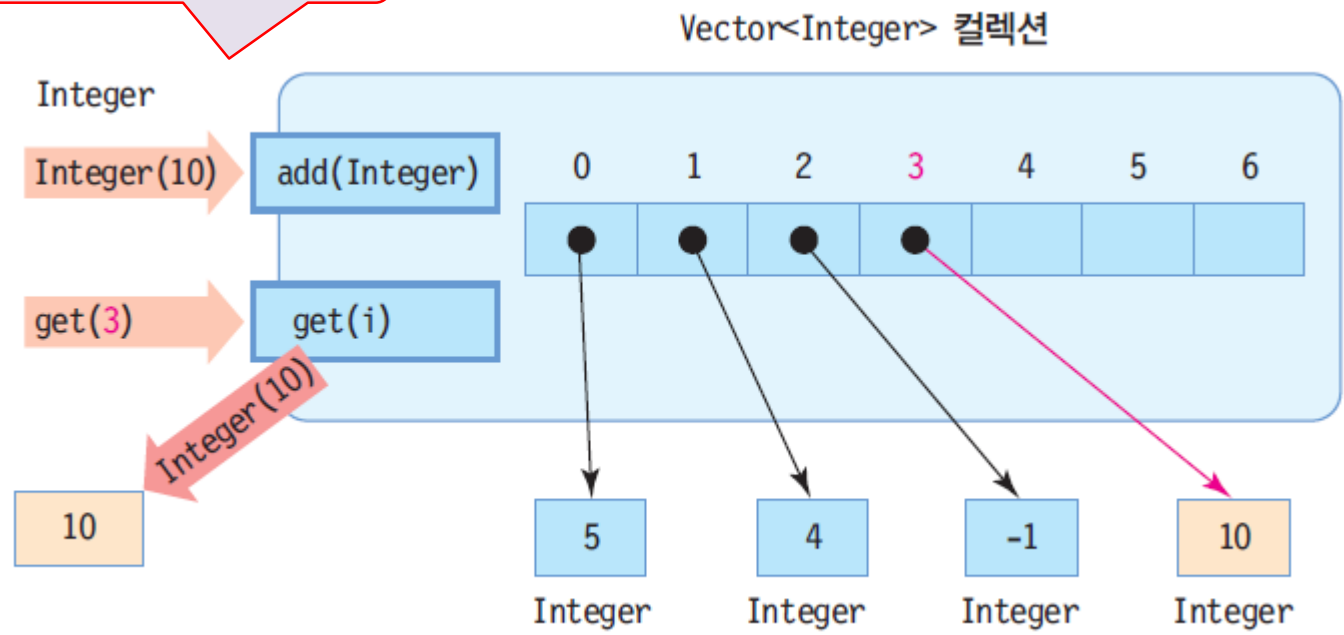
- ▷ Vector 또한 가변적으로 멤버 객체를 추가할 수 있음
- ▷ ArrayList와 달리 Vector에서는 관리 멤버 객체를 Element 즉, 요소라고도 부름
- ▷ Vector에는 저장 용량(capacity)이라는 개념이 존재하는데, 네 번째 생성자에서 입력 받는 두 개의 매개변수 중 첫 번째는 Vector **객체 용량의 초깃값**을, 두 번째는 **증가값**을 설정

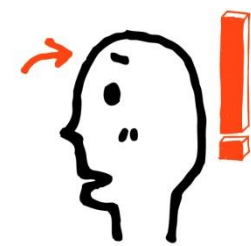


Vector<Integer> 컬렉션 내부 구성

```
Vector<Integer> v = new Vector<Integer>();
```

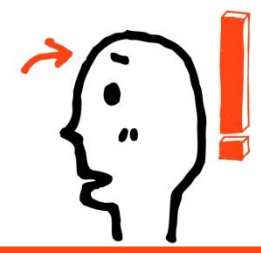
add()를 이용하여 요소를 삽입하고
get()을 이용하여 요소를 검색합니다





타입 매개 변수 사용하지 않는 경우 경고 발생

Vector<Integer>나 Vector<String> 등 타입 매개 변수를 사용하여야 함



Vector<E> 클래스의 주요 메소드

메소드	설명
boolean add(E element)	벡터의 맨 뒤에 element 추가
void add(int index, E element)	인덱스 index에 element를 삽입
int capacity()	벡터의 현재 용량 리턴
boolean addAll(Collection<? extends E> c)	컬렉션 c의 모든 요소를 벡터의 맨 뒤에 추가
void clear()	벡터의 모든 요소 삭제
boolean contains(Object o)	벡터가 지정된 객체 o를 포함하고 있으면 true 리턴
E elementAt(int index)	인덱스 index의 요소 리턴
E get(int index)	인덱스 index의 요소 리턴
int indexOf(Object o)	o와 같은 첫 번째 요소의 인덱스 리턴. 없으면 -1 리턴
boolean isEmpty()	벡터가 비어 있으면 true 리턴
E remove(int index)	인덱스 index의 요소 삭제
boolean remove(Object o)	객체 o와 같은 첫 번째 요소를 벡터에서 삭제
void removeAllElements()	벡터의 모든 요소를 삭제하고 크기를 0으로 만듦
int size()	벡터가 포함하는 요소의 개수 리턴
Object[] toArray()	벡터의 모든 요소를 포함하는 배열 리턴



Vector<E> 클래스의 활용 예

벡터 생성

```
Vector<Integer> v = new Vector<Integer>(7);
```

v

Vector<Integer> 객체

요소 삽입

```
v.add(5);  
v.add(new Integer(4));  
v.add(-1);
```

v

요소 개수 n
벡터의 용량 c

```
int n = v.size(); // n은 3  
int c = v.capacity(); // c는 7
```

n = 3
c = 7

요소 중간 삽입

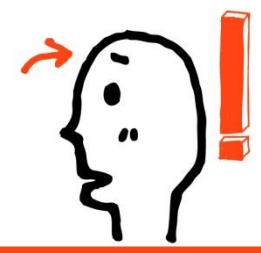
```
v.add(2, 100);  
  
v.add(5, 100);  
// v.size()보다 큰 곳에 삽입 불가능, 오류
```

v

요소 얻어내기

```
Integer obj = v.get(1);  
int i = obj.intValue();
```

obj

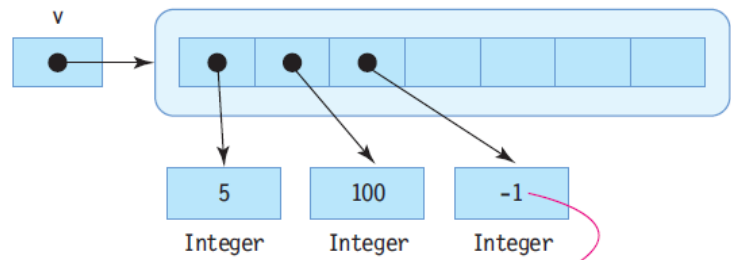


Vector<E> 클래스의 활용 예

요소 삭제

```
v.remove(1);
```

```
v.remove(4);  
// 인덱스 4에 요소 객체가 없으므로 오류
```



마지막 요소

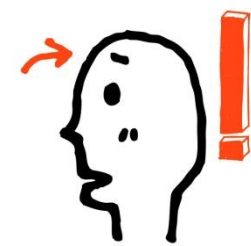
```
int last = v.lastElement();
```

last = -1

모든 요소 삭제

```
v.removeAllElements();
```





컬렉션과 자동 박싱/언박싱

- JDK 1.5 이전
 - ▣ 기본 타입 데이터를 Wrapper 클래스를 이용하여 객체로 만들어 사용

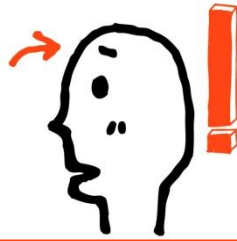
```
Vector<Integer> v = new Vector<Integer>();  
v.add(new Integer(4));  
v.add(new Character('r'));  
v.add(new Double(3.14));
```

- ▣ 컬렉션으로부터 요소를 얻어올 때, Wrapper 클래스로 캐스팅 필요

```
Integer n = (Integer)v.get(0);  
int k = n.intValue(); // k = 4
```

- JDK 1.5부터
 - ▣ 자동 박싱/언박싱의 기능 추가

```
Vector<Integer> v = new Vector<Integer> ();  
v.add(4); // 4 → new Integer(4)로 자동 박싱  
int k = v.get(0); // Integer 타입이 int 타입으로 자동 언박싱, k = 4
```



정수 값만 다루는 Vector<Integer>

정수 값만 다루는 제네릭 벡터를 생성하고 활용하는 사례를 보인다.
다음 코드에 대한 결과는 무엇인가?

```
import java.util.Vector;

public class VectorEx {
    public static void main(String[] args) {
        // 정수 값만 다루는 제네릭 벡터 생성
        Vector<Integer> v = new Vector<Integer>();

        v.add(5); // 5 삽입
        v.add(4); // 4 삽입
        v.add(-1); // -1 삽입

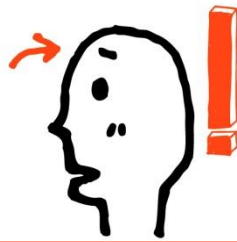
        // 벡터 중간에 삽입하기
        v.add(2, 100); // 4와 -1 사이에 정수 100 삽입

        System.out.println("벡터 내의 요소 객체 수 : " + v.size());
        System.out.println("벡터의 현재 용량 : " + v.capacity());

        // 모든 요소 정수 출력하기
        for(int i=0; i<v.size(); i++) {
            int n = v.get(i);
            System.out.println(n);
        }
    }
}
```

```
// 벡터 속의 모든 정수 더하기
int sum = 0;
for(int i=0; i<v.size(); i++) {
    int n = v.elementAt(i);
    sum += n;
}
System.out.println("벡터에 있는 정수 합 : "
                    + sum);
}
```

```
5
4
100
-1
벡터에 있는 정수 합 : 108
```



Point 클래스의 객체들만 저장하는 벡터 만들기

(x, y) 한 점을 추상화한 Point 클래스를 만들고
Point 클래스의 객체만 저장하는 벡터를 작성하라.

```
import java.util.Vector;

class Point {
    private int x, y;
    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

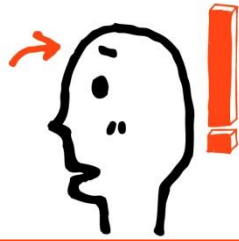
    public String toString() {
        return "(" + x + "," + y + ")";
    }
}
```

```
public class PointVectorEx {
    public static void main(String[] args) {
        // Point 객체를 요소로만 가지는 벡터 생성
        Vector<Point> v = new Vector<Point>();

        // 3 개의 Point 객체 삽입
        v.add(new Point(2, 3));
        v.add(new Point(-5, 20));
        v.add(new Point(30, -8));

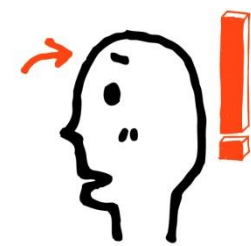
        // 벡터에 있는 Point 객체 모두 검색하여 출력
        for(int i=0; i<v.size(); i++) {
            Point p = v.get(i); // 벡터에서 i 번째 Point 객체 얻어내기
            System.out.println(p); // p.toString()을 이용하여 객체 p 출력
        }
    }
}
```

```
(2,3)
(-5,20)
(30,-8)
```

ArrayList<E>

- ArrayList<E>의 특성
 - ▣ java.util.ArrayList, 가변 크기 배열을 구현한 클래스
 - <E>에서 E 대신 요소로 사용할 특정 타입으로 구체화(제네릭)
 - ▣ ArrayList에 삽입 가능한 것
 - 객체(인스턴스), 객체, null
 - 기본 타입은 박싱/언박싱으로 Wrapper 객체로 만들어 저장
 - ▣ ArrayList에 객체 삽입/삭제
 - 리스트의 맨 뒤에 객체 추가
 - 리스트의 중간에 객체 삽입
 - 임의의 위치에 있는 객체 삭제 가능



ArrayList<E>

▶ List 인터페이스를 구현하는 ArrayList와 Vector 클래스

▷ ArrayList 객체를 생성하는 방법

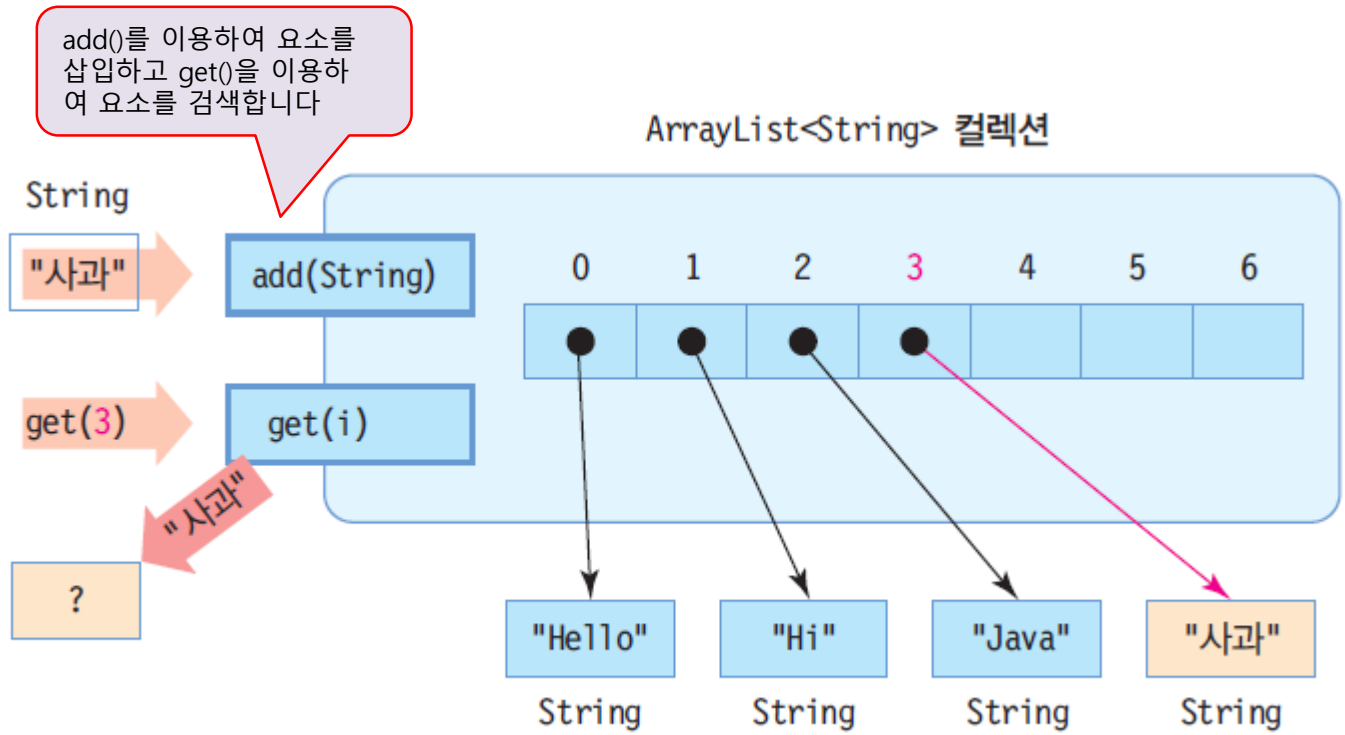
```
사용법 : ArrayList<[[데이터형]]> [변수 이름] = new ArrayList <[[데이터형]]> ( );  
        ArrayList<[[데이터형]]> [변수 이름] = new ArrayList <[[데이터형]]> (Collection c);  
        ArrayList<[[데이터형]]> [변수 이름] = new ArrayList <[[데이터형]]> ([int 초깃값]);  
사용예 : ArrayList<String> strList1 = new ArrayList<String>();  
        ArrayList<String> strList2 = new ArrayList<String>(strList1);  
        ArrayList<String> strList3 = new ArrayList<String>(30);
```

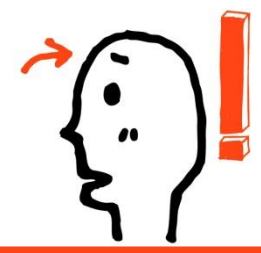
- ▷ 선언된 strList1 객체는 ArrayList 객체를 간단하게 생성하는 방법
- ▷ strList1을 포함하는 새로운 ArrayList strList2 객체를 생성하는 방법
- ▷ strList3 객체는 매개변수 값만큼 초기에 포함할 수 있는 메모리를 확보한 뒤 객체를 생성하는 방법



ArrayList<String> 컬렉션의 내부 구성

```
ArrayList<String> = new ArrayList<String>();
```





ArrayList<E> 클래스의 주요 메소드

메소드	설명
boolean add(E element)	ArrayList의 맨 뒤에 element 추가
void add(int index, E element)	인덱스 index에 지정된 element 삽입
boolean addAll(Collection<? extends E> c)	컬렉션 c의 모든 요소를 ArrayList의 맨 뒤에 추가
void clear()	ArrayList의 모든 요소 삭제
boolean contains(Object o)	ArrayList가 지정된 객체를 포함하고 있으면 true 리턴
E elementAt(int index)	index 인덱스의 요소 리턴
E get(int index)	index 인덱스의 요소 리턴
int indexOf(Object o)	o와 같은 첫 번째 요소의 인덱스 리턴. 없으면 -1 리턴
boolean isEmpty()	ArrayList가 비어 있으면 true 리턴
E remove(int index)	index 인덱스의 요소 삭제
boolean remove(Object o)	o와 같은 첫 번째 요소를 ArrayList에서 삭제
int size()	ArrayList가 포함하는 요소의 개수 리턴
Object[] toArray()	ArrayList의 모든 요소를 포함하는 배열 리턴



예를 통한 ArrayList<E>의 이해

ArrayList 생성

```
ArrayList<String> a = new ArrayList<String>(7);
```

a

요소 삽입

```
a.add("Hello");  
a.add("Hi");  
a.add("Java");
```

요소 개수 n
용량

```
int n = a.size(); // n은 3  
int c = a.capacity(); // capacity() 메소드 없음
```

n = 3

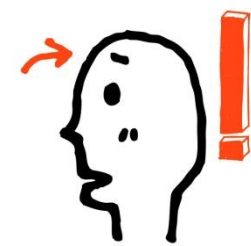
요소 중간 삽입

```
a.add(2, "Sahni");  
  
a.add(5, "Sahni");  
// a.size()보다 큰 위치에 삽입 불가능, 오류
```

요소 알아내기

```
String str = a.get(1);
```

"Hi"

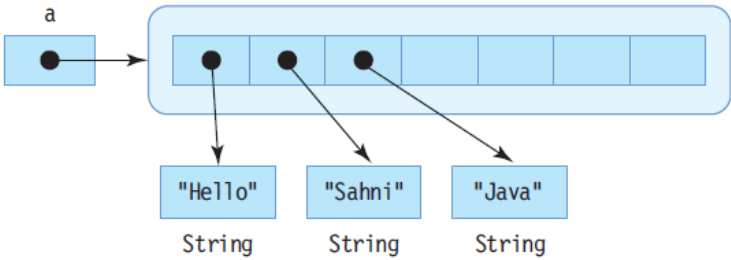


예를 통한 ArrayList<E>의 이해

요소 삭제

```
a.remove(1);
```

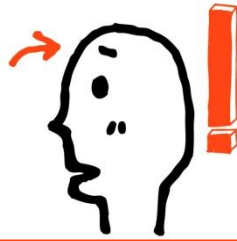
```
a.remove(4); // 오류
```



모든 요소 삭제

```
a.clear();
```





ArrayList에 문자열 담기

키보드로 문자열을 입력 받아 ArrayList에 삽입하고 가장 긴 이름을 출력하라.

```
import java.util.*;

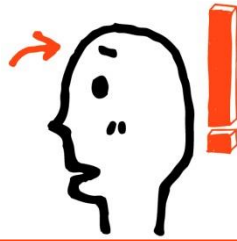
public class ArrayListEx {
    public static void main(String[] args) {
        // 문자열만 삽입가능한 ArrayList 컬렉션 생성
        ArrayList<String> a = new ArrayList<String>();

        // 키보드로부터 4개의 이름 입력받아 ArrayList에 삽입
        Scanner scanner = new Scanner(System.in);
        for(int i=0; i<4; i++) {
            System.out.print("이름을 입력하세요>>");
            String s = scanner.next(); // 키보드로부터 이름 입력
            a.add(s); // ArrayList 컬렉션에 삽입
        }

        // ArrayList에 들어 있는 모든 이름 출력
        for(int i=0; i<a.size(); i++) {
            // ArrayList의 i 번째 문자열 얻어오기
            String name = a.get(i);
            System.out.print(name + " ");
        }
    }
}
```

```
// 가장 긴 이름 출력
int longestIndex = 0;
for(int i=1; i<a.size(); i++) {
    if(a.get(longestIndex).length() < a.get(i).length())
        longestIndex = i;
}
System.out.println("\n가장 긴 이름은 : " +
    a.get(longestIndex));
}
```

```
이름을 입력하세요>>Mike
이름을 입력하세요>>Jane
이름을 입력하세요>>Ashley
이름을 입력하세요>>Helen
Mike Jane Ashley Helen
가장 긴 이름은 : Ashley
```

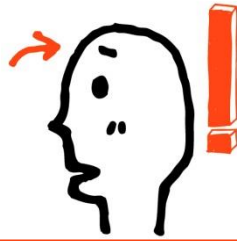


예를 통한 ArrayList와 Vector 클래스의 이해

▶ List 인터페이스를 구현하는 ArrayList와 Vector 클래스

코드 7-5 package com.gilbut.chapter7;

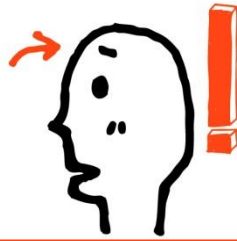
```
1  import java.util.ArrayList;
2  import java.util.List;
3  import java.util.Random;
4  import java.util.Vector;
5
6  public class ListExample1
7  {
8      private final int MAX_INT = 9;
9      private Random random = new Random();
10     private ArrayList<Integer> arrayList = new ArrayList<Integer>();
11     private Vector<Integer> vector = new Vector<Integer>();
12
13     public static void main(String[] args)
14     {
15         ListExample1 example = new ListExample1();
16         example.testArrayList();
17         example.testVector();
18     }
19
```

예를 통한 ArrayList와 Vector 클래스의 이해

▶ List 인터페이스를 구현하는 ArrayList와 Vector 클래스

```
20     public void testArrayList()
21     {
22         // arrayList 초깃값 설정
23         for (int i = 0; i < MAX_INT; i++)
24         {
25             arrayList.add(random.nextInt(MAX_INT));
26         }
27
28         this.printMember(arrayList);
29         arrayList.remove(2);
30         this.printMember(arrayList);
31         arrayList.add(8);
32         this.printMember(arrayList);
33         arrayList.clear();
34         this.printMember(arrayList);
35     }
36
```



예를 통한 ArrayList와 Vector 클래스의 이해

▶ List 인터페이스를 구현하는 ArrayList와 Vector 클래스

```
37     public void testVector()
38     {
39         // vector 초기값 설정
40         for (int i = 0; i < MAX_INT; i++)
41         {
42             vector.add(random.nextInt(MAX_INT));
43         }
44
45         this.printMember(vector);
46         vector.remove(2);
47         this.printMember(vector);
48         vector.add(8);
49         this.printMember(vector);
50         vector.clear();
51         this.printMember(vector);
52     }
53
```

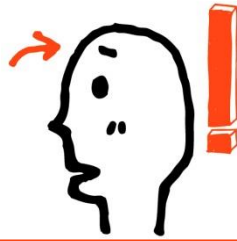


예를 통한 ArrayList와 Vector 클래스의 이해

▶ List 인터페이스를 구현하는 ArrayList와 Vector 클래스

```
54 public void printMember(List<Integer> list)
55 {
56     int totalSize = list.size();
57
58     if (list instanceof ArrayList)
59         System.out.print("ArrayList Member (" + totalSize + ") : ");
60     else if (list instanceof Vector)
61         System.out.print("Vector Member (" + totalSize + ") : ");
62     else
63         System.out.print("Unknown Member (" + totalSize + ") : ");
64
65     // 강화된 for 구문을 이용하여 멤버 객체 확인
66     for (Integer item : list)
67     {
68         System.out.print "[" + item + " ] ";
69     }
70     System.out.print("\n");
71 }
72 }
```

ArrayList나 Vector 클래스 모두 List로 구현한 구현 클래스이므로, ArrayList 객체나 Vector 객체를 매개변수로 받을 수 있습니다.



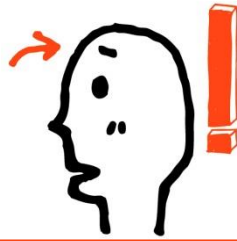
예를 통한 ArrayList와 Vector 클래스의 이해

▶ List 인터페이스를 구현하는 ArrayList와 Vector 클래스

실행결과

```
ArrayList Member (9) : [8] [3] [0] [7] [4] [4] [3] [0] [1]
//remove(2)로 인한 [0] 삭제
ArrayList Member (8) : [8] [3] [7] [4] [4] [3] [0] [1]
//add(8)로 인한 [8] 추가
ArrayList Member (9) : [8] [3] [7] [4] [4] [3] [0] [1] [8]
//clear()로 인한 모든 객체 삭제
ArrayList Member (0) :
Vector Member (9) : [0] [1] [3] [5] [1] [6] [0] [4] [2]
Vector Member (8) : [0] [1] [5] [1] [6] [0] [4] [2]
Vector Member (9) : [0] [1] [5] [1] [6] [0] [4] [2] [8]
Vector Member (0) :
```

- ▶ printMember() 메소드는 ArrayList 클래스와 Vector 클래스의 멤버 변수들을 출력하기 위해서 공용
- ▶ 매개변수로서 Integer형의 List 인터페이스 객체를 입력받음

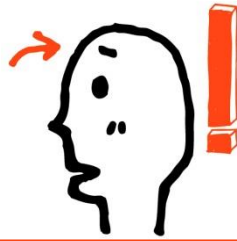


예를 통한 ArrayList와 Vector 클래스의 이해

▶ List 인터페이스를 구현하는 ArrayList와 Vector 클래스

코드 7-6 package com.gilbut.chapter7;

```
1  import java.util.Vector;
2
3  public class VectorExample1
4  {
5      // count 변수는 printStatus() 메소드의 호출 횟수를 카운트한다.
6      private int count = 0;
7      // 초기 capacity = 10으로 증가 capacity = 5로 선언
8      private Vector<String> vector = new Vector<String>(10, 5);
9
10     public static void main(String[] args)
11     {
12         VectorExample1 example = new VectorExample1();
13         example.execute();
14     }
15
```

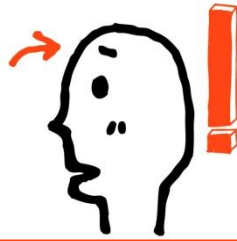


예를 통한 ArrayList와 Vector 클래스의 이해

▶ List 인터페이스를 구현하는 ArrayList와 Vector 클래스

```
16 public void execute()
17 {
18     //첫 번째 printStatus() 호출
19     this.printStatus();
20     for (int i = 0; i < 9; i++)
21     {
22         vector.addElement(String.valueOf(i));
23     }
24
25     //두 번째 printStatus() 호출
26     this.printStatus();
27     for (int i = 0; i < 4; i++)
28     {
29         vector.add(String.valueOf(i));
30     }
31 }
```

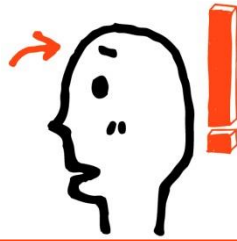
Vector 클래스의 메소드 사용법과 Vector 객체 Size 변화에 주의하세요



예를 통한 ArrayList와 Vector 클래스의 이해

▶ List 인터페이스를 구현하는 ArrayList와 Vector 클래스

```
32         //세 번째 printStatus() 호출
33         this.printStatus();
34         vector.trimToSize();
35
36         //네 번째 printStatus() 호출
37         this.printStatus();
38         vector.setSize(20);
39
40         //다섯 번째 printStatus() 호출
41         this.printStatus();
42
43         System.out.println(vector.toString());
44     }
45
46     public void printStatus()
47     {
48         count++;
49         System.out.println "[" + count + " ] Capacity : " + vector.capacity() + ",
         Element size : " + vector.size());
50     }
51 }
```



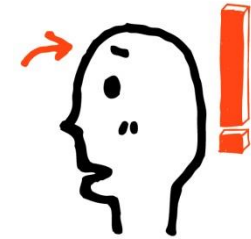
예를 통한 ArrayList와 Vector 클래스의 이해

▶ List 인터페이스를 구현하는 ArrayList와 Vector 클래스

실행결과

```
[1] Capacity : 10, Element size : 0  
[2] Capacity : 10, Element size : 9  
[3] Capacity : 15, Element size : 13  
[4] Capacity : 13, Element size : 13  
[5] Capacity : 20, Element size : 20  
[0, 1, 2, 3, 4, 5, 6, 7, 8, 0, 1, 2, 3, null, null, null, null, null, null]
```

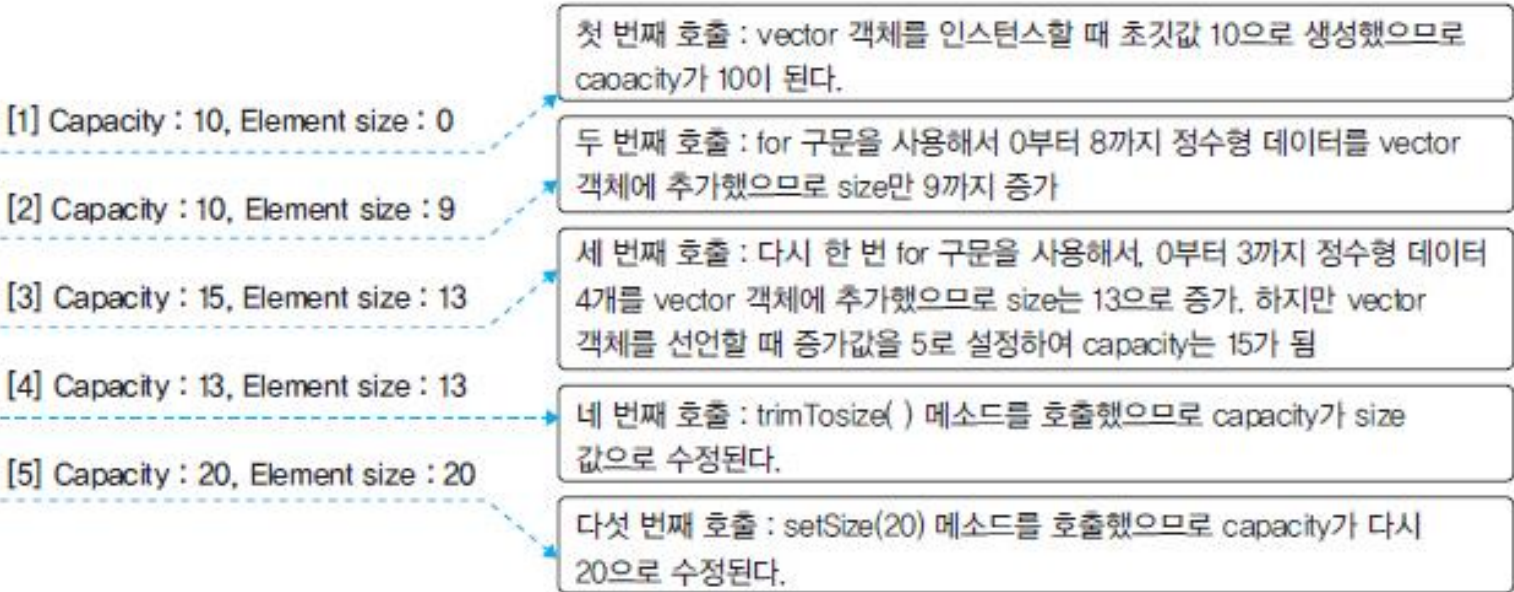
- ▶ `printStatus()` 메소드는 호출될 때마다, vector 객체의 용량(capacity)과 요소 개수(size)를 화면에 출력하는 역할
- ▶ 여러 번 호출되면 count 변수를 증가시켜 호출된 순번을 화면에 출력

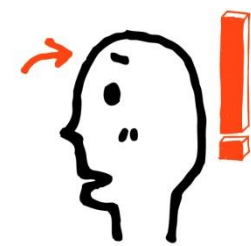


예를 통한 ArrayList와 Vector 클래스의 이해

▶ List 인터페이스를 구현하는 ArrayList와 Vector 클래스

▶ toString() 메소드는 객체의 정보를 편리하게 볼 수 있도록 **오버라이딩**해서 사용





List 인터페이스를 구현

▶ List 인터페이스를 구현하는 ArrayList와 Vector 클래스

- ▶ ArrayList와 Vector 클래스의 가장 큰 차이점은 동기화의 지원 여부
- ▶ ArrayList와 Vector는 컬렉션 클래스이므로 객체를 생성할 때는 new 키워드를 사용해서 인스턴스화 해야 됨
- ▶ 인스턴스화되지 않는 객체에 메소드 혹은 클래스 변수 호출 시 NullPointerException 에러 발생

공통점	차이점
• 동적으로 크기 변경이 가능하다. • 데이터를 중복해서 저장할 수 있다. • 멤버 객체의 순서가 있는 컬렉션 클래스다.	• Vector는 동기화(Synchronization)를 제공하지만 ArrayList는 동기화를 제공하지 않는다. • ArrayList가 Vector보다 속도가 빠르다.



ArrayList<E>, LinkedList<E>

- ✓ List<E> 인터페이스를 구현하는 대표적인 제네릭 클래스
 - ➔ ArrayList<E>, LinkedList<E>
- ✓ List<E> 인터페이스를 구현 클래스의 인스턴스 저장 특징
 - ➔ 동일한 인스턴스의 중복 저장을 허용한다.
 - ➔ 인스턴스의 저장 순서가 유지된다.

```
public static void main(String[] args)
{
    ArrayList<Integer> list=new ArrayList<Integer>();
    /* 데이터의 저장 */
    list.add(new Integer(11));
    list.add(new Integer(22));
    list.add(new Integer(33));

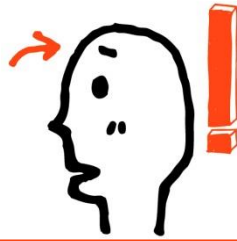
    /* 데이터의 참조 */
    System.out.println("1차 참조");
    for(int i=0; i<list.size(); i++)
        System.out.println(list.get(i));    // 0이 첫 번째

    /* 데이터의 삭제 */
    list.remove(0);    // 0이 전달되었으므로 첫 번째 데이터 삭제
    System.out.println("2차 참조");
    for(int i=0; i<list.size(); i++)
        System.out.println(list.get(i));
}
```

ArrayList<E>는 이름이 의미하듯이 배열 기반으로 데이터를 저장한다.

실행 결과

```
1차 참조
11
22
33
2차 참조
22
33
```



LinkedList<E>

□ LinkedList<E>의 특성

▣ java.util.LinkedList

- E에 요소로 사용할 타입 지정하여 구체와

▣ List 인터페이스를 구현한 컬렉션 클래스

▣ Vector, ArrayList 클래스와 매우 유사하게 작동

▣ 요소 객체들은 양방향으로 연결되어 관리됨

▣ 요소 객체는 맨 앞, 맨 뒤에 추가 가능

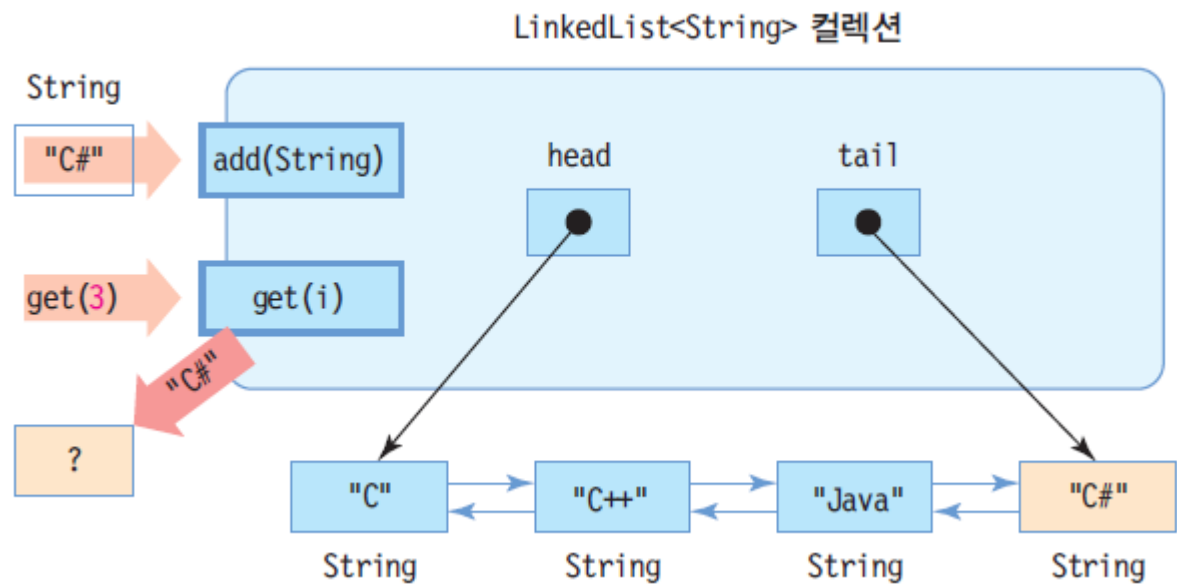
▣ 요소 객체는 인덱스를 이용하여 중간에 삽입 가능

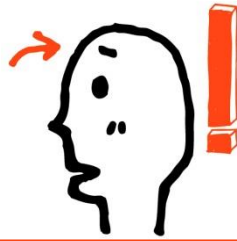
▣ 맨 앞이나 맨 뒤에 요소를 추가하거나 삭제할 수 있어 스택이나 큐로 사용 가능



LinkedList<String>의 내부 구성과 put(), get() 메소드

```
LinkedList<String> l = new LinkedList<String>();
```





ArrayList<E>, LinkedList<E>

✓ 데이터의 저장방식

➔ 이름이 의미하듯이 '리스트'라는 자료구조를 기반으로 데이터를 저장한다.

✓ 사용방법

➔ ArrayList<E>의 사용방법과 거의 동일하다! 다만, 데이터를 저장하는 방식에서 큰 차이가 있을 뿐이다.

➔ 대부분의 경우 ArrayList<E>를 대체할 수 있다.

```
public static void main(String[] args)
{
    LinkedList<Integer> list=new LinkedList<Integer>();

    /* 데이터의 저장 */
    list.add(new Integer(11));
    list.add(new Integer(22));
    list.add(new Integer(33));

    /* 데이터의 참조 */
    System.out.println("1차 참조");
    for(int i=0; i<list.size(); i++)
        System.out.println(list.get(i));

    /* 데이터의 삭제 */
    list.remove(0);
    System.out.println("2차 참조");
    for(int i=0; i<list.size(); i++)
        System.out.println(list.get(i));
}
```

1차 참조 **실행 결과**

11

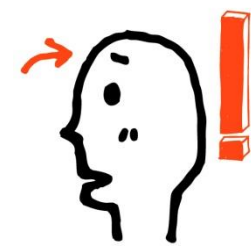
22

33

2차 참조

22

33



ArrayList<E>와 LinkedList<E>의 차이점

ArrayList<E>의 특징, 배열의 특징과 일치한다.

- 저장소의 용량을 늘리는 과정에서 많은 시간이 소요된다.
- 데이터의 삭제에 필요한 연산과정이 매우 길다.
- 데이터의 참조가 용이해서 빠른 참조가 가능하다.

ArrayList<E>의 **단점**

ArrayList<E>의 **단점**

ArrayList<E>의 **장점**

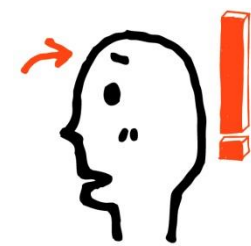
LinkedList<E>의 특징, 리스트 자료구조의 특징과 일치한다.

- 저장소의 용량을 늘리는 과정이 간단하다.
- 데이터의 삭제가 매우 간단하다.
- 데이터의 참조가 다소 불편하다.

LinkedList<E>의 **장점**

LinkedList<E>의 **장점**

LinkedList<E>의 **단점**



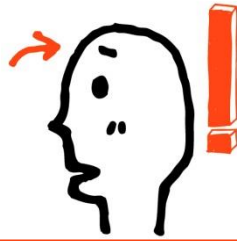
Iterator를 이용한 인스턴스의 순차적 접근

Iterator<E> 인터페이스

- Collection<E> 인터페이스에는 iterator라는 이름의 메소드가 다음의 형태로 정의
→ Iterator<E> iterator() { }
- iterator 메소드가 반환하는 참조 값의 인스턴스는 Iterator<E> 인터페이스를 구현하고 있다.
- iterator 메소드가 반환한 참조 값의 인스턴스를 이용하면, 컬렉션 인스턴스에 저장된 인스턴스의 순차적 접근이 가능함.
- iterator 메소드의 반환형이 Iterator<E>이니, 반환된 참조 값을 이용해서 Iterator<E>에 선언된 함수들만 호출하면 된다.

Iterator<E> 인터페이스에 정의된 메소드

- | | |
|----------------------|-------------------------------------|
| • boolean hasNext() | 참조할 다음 번 요소(element)가 존재하면 true를 반환 |
| • E next() | 다음 번 요소를 반환 |
| • void remove() | 현재 위치의 요소를 삭제 |



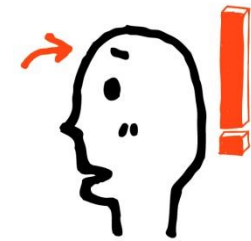
Iterator를 이용한 인스턴스의 순차적 접근

▶ 반복 처리를 위한 Iterator

▷ Iterator 객체를 받기 위한 iterator() 메소드의 사용법

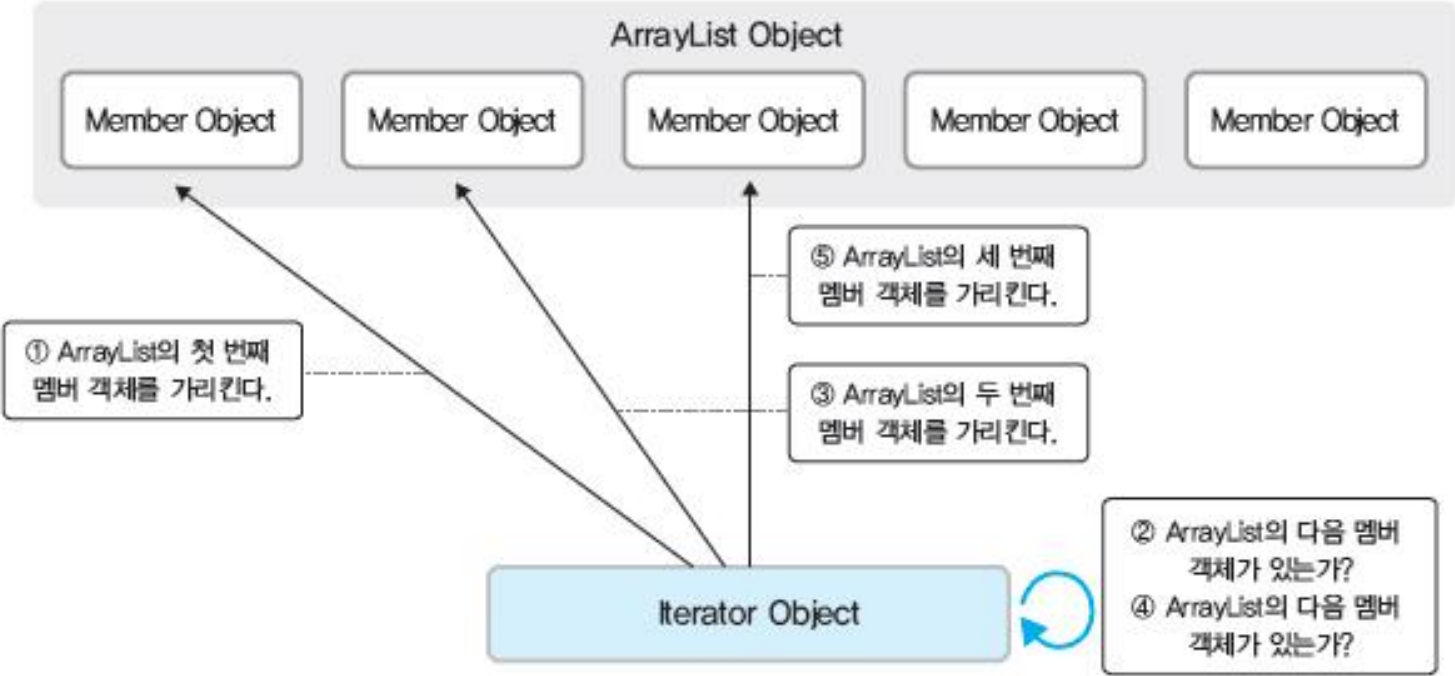
```
사용법 : public Iterator<E> iterator( ) //선언부
        Iterator< [멤버 객체의 데이터형] > [변수 이름] = [Collection 객체].iterator( );
사용예 : ArrayList<String> strList = new ArrayList<String>();
        Iterator iter = strList.iterator();
        HashSet<Integer> intSet = new HashSet<Integer>();
        Iterator iter = intSet.iterator();
```

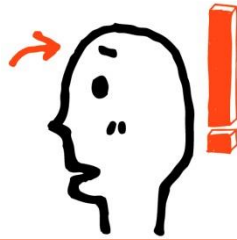
- ▷ Iterator는 복잡한 자료구조 대신에 오직 반복을 위한 기능을 제공
- ▷ 컬렉션 객체의 기능을 버린 새로운 객체



Iterator를 이용한 인스턴스의 순차적 접근

▶ Iterator의 동작 원리





Iterator의 사용 예

```
public static void main(String[] args)
{
    LinkedList<String> list=new LinkedList<String>();
    list.add("First");
    list.add("Second");
    list.add("Third");
    list.add("Fourth");

    Iterator<String> itr=list.iterator();

    System.out.println("반복자를 이용한 1차 출력과 \"Third\" 삭제");
    while(itr.hasNext())
    {
        String curStr=itr.next();
        System.out.println(curStr);
        if(curStr.compareTo("Third")==0)
            itr.remove();
    }

    System.out.println("\n\"Third\" 삭제 후 반복자를 이용한 2차 출력 ");
    itr=list.iterator();
    while(itr.hasNext())
        System.out.println(itr.next());
}
```

iterator 메소드가 생성하는 인스턴스를

Iterator<String> itr=list.iterator(); 가리켜 '반복자'라 한다.

실행 결과

반복자를 이용한 1차 출력과 "Third" 삭제

First
Second
Third
Fourth

"Third" 삭제 후 반복자를 이용한 2차 출력

First
Second
Fourth



'반복자'를 사용하는 이유

- 반복자를 사용하면, 컬렉션 클래스의 종류에 상관없이 동일한 형태의 데이터 참조방식을 유지할 수 있다.
- 따라서 컬렉션 클래스의 교체에 큰 영향이 없다.
- 컬렉션 클래스 별 데이터 참조방식을 별도로 확인할 필요가 없다.

```
public static void main(String[] args)
{
    LinkedList<String> list=new LinkedList<String>();
    list.add("First");
    list.add("Second");
    list.add("Third");
    list.add("Fourth");

    Iterator<String> itr=list.iterator();

    System.out.println("반복자를 이용한 1차 출력과 \"Third\" 삭제");
    while(itr.hasNext())
    {
        String curStr=itr.next();
        System.out.println(curStr);
        if(curStr.compareTo("Third")==0)
            itr.remove();
    }

    System.out.println("\n\"Third\" 삭제 후 반복자를 이용한 2차 출력 ");
    itr=list.iterator();
    while(itr.hasNext())
        System.out.println(itr.next());
}
```

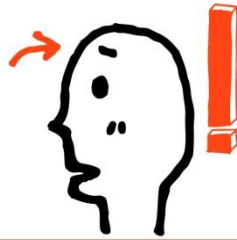
원편은 앞서 소개한 예제이다. 그런데, 이 예제는 반복자를 사용했기 때문에, LinkedList<E>가 어울리지 않아서, 컬렉션 클래스를 HashSet<E>로 변경해야 할 때, 다음과 같이 변경이 매우 용이하다.

변경의 전부!

LinkedList<String> list
=new LinkedList<String>();

↓

HashSet<String> set
=new HashSet<String>();

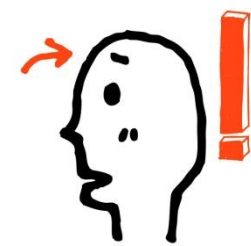


예제를 통한 Iterator의 이해

▶ 반복 처리를 위한 Iterator

코드 7-10 package com.gilbut.chapter7;

```
1  import java.util.ArrayList;
2  import java.util.Iterator;
3
4  public class IteratorExample1
5  {
6      private ArrayList<String> urlList = new ArrayList<String>();
7
8      public static void main(String[] args)
9      {
10         IteratorExample1 example = new IteratorExample1();
11         example.init();
12         example.execute();
13     }
14
15     public void init()
16     {
17         urlList.add("http://www.daum.net");
18         urlList.add("http://www.google.co.kr");
19         urlList.add("http://www.facebook.com");
20     }
21 }
```

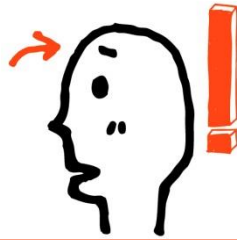


예제를 통한 Iterator의 이해

▶ 반복 처리를 위한 Iterator

```
22 public void execute()
23 {
24     for (int i = 0; i < urlList.size(); i++)
25     {
26         System.out.println("URL : " + urlList.get(i));
27     }
28     System.out.println("-----");
29
30     for (String url : urlList)
31     {
32         System.out.println("URL : " + url);
33     }
34     System.out.println("-----");
35
36     Iterator<String> iter = urlList.iterator();
37     while (iter.hasNext())
38     {
39         System.out.println("URL : " + (String) iter.next());
40     }
41 }
42 }
```

ArrayList 객체의 멤버 변수를 다루는 세 가지 방법을 보여주는 메소드. 아래의 Iterator 사용법과 for 구문을 사용한 다른 방법을 비교해 볼까요?



예제를 통한 Iterator의 이해

▶ 반복 처리를 위한 Iterator

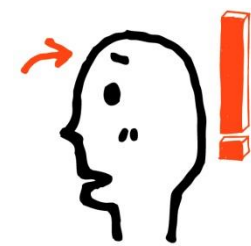
실행결과

```
URL : http://www.daum.net  
URL : http://www.google.co.kr  
URL : http://www.facebook.com
```

```
URL : http://www.daum.net  
URL : http://www.google.co.kr  
URL : http://www.facebook.com
```

```
URL : http://www.daum.net  
URL : http://www.google.co.kr  
URL : http://www.facebook.com
```

- ▶ ArrayList 객체인 urlList의 내부 객체를 반복 처리하는 세 가지 방법에 대한 내용



컬렉션 클래스를 이용한 정수의 저장

`ArrayList<int> arr=new ArrayList<int>();``error`

`LinkedList<int> link=new LinkedList<int>();``error`

기본 자료형 정보를 이용해서 제네릭 인스턴스 생성 불가능! 따라서 Wrapper 클래스를 기반으로 컬렉션 인스턴스를 생성한다.

```
public static void main(String[] args)
{
    LinkedList<Integer> list=new LinkedList<Integer>();
    list.add(10);           // Auto Boxing
    list.add(20);           // Auto Boxing
    list.add(30);           // Auto Boxing

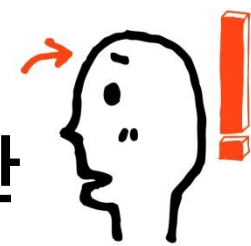
    Iterator<Integer> itr=list.iterator();

    while(itr.hasNext())
    {
        int num=itr.next();    // Auto Unboxing
        System.out.println(num);
    }
}
```

실행 결과

10
20
30

Auto Boxing과 Auto Unboxing
의 도움으로 정수 단위의 데이터
입출력이 매우 자연스럽다!



배열보다는 컬렉션 인스턴스가 좋다. : 컬렉션 변환

다음 두 가지 이유로 배열보다 ArrayList<E>가 더 좋다.

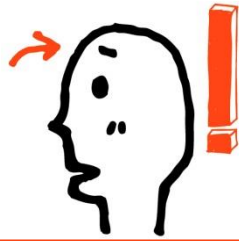
인스턴스의 저장과 삭제가 편하다.

반복자를 쓸 수 있다.

단, 배열처럼 선언과 동시에 초기화가 불가능하다. 그러나 다음 방법을 쓸 수 있다.

```
List<String> list = Arrays.asList("Toy", "Robot", "Box");
```

- 인자로 전달된 인스턴스들을 저장한 컬렉션 인스턴스의 생성 및 반환
- 이렇게 생성된 리스트 인스턴스는 Immutable 인스턴스이다.



배열보다는 컬렉션 인스턴스가 좋다. : 이어서

다음 생성자를 통해서 새로운 ArrayList 인스턴스 생성 가능

```
public ArrayList(Collection<? extends E> c) {...}
```

- Collection<E>를 구현한 컬렉션 인스턴스를 인자로 전달받는다.
- 그리고 E는 인스턴스 생성 과정에서 결정되므로 무엇이든 될 수 있다.
- 덧붙여서 매개변수 c로 전달된 컬렉션 인스턴스에서는 참조만(꺼내기만) 가능하다.

```
public static void main(String[] args) {
```

```
    List<String> list = Arrays.asList("Toy", "Box", "Robot", "Box");
```

```
    // 생성자 public ArrayList(Collection<? extends E> c)를 통한 인스턴스 생성
```

```
    list = new ArrayList<>(list);
```

```
    ....
```

```
}
```

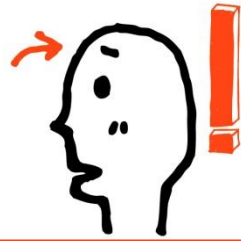


배열 기반 리스트를 연결 기반 리스트로...

```
public ArrayList(Collection<? extends E> c)    // ArrayList<E> 생성자 중 하나
→ 인자로 전달된 컬렉션 인스턴스로부터 ArrayList<E> 인스턴스 생성
```

```
public LinkedList(Collection<? extends E> c)    // LinkedList<E> 생성자 중 하나
→ 인자로 전달된 인스턴스로부터 LinkedList<E> 인스턴스 생성
```

```
public static void main(String[] args) {
    List<String> list = Arrays.asList("Toy", "Box", "Robot", "Box");
    list = new ArrayList<>(list);
    . . .
    // ArrayList<E> 인스턴스 기반으로 LinkedList<E> 인스턴스 생성
    list = new LinkedList<>(list);
    . . .
}
```



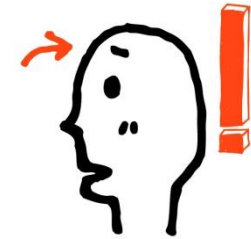
기본 자료형 데이터의 저장과 참조

```
public static void main(String[] args) {  
    LinkedList<Integer> list = new LinkedList<>();  
    list.add(10);    // 저장 과정에서 오토 박싱 진행  
    list.add(20);  
    list.add(30);  
  
    int n;  
    for(Iterator<Integer> itr = list.iterator(); itr.hasNext(); ) {  
        n = itr.next();    // 오토 언박싱 진행  
        System.out.print(n + "\t");  
    }  
    System.out.println();  
}
```



Collections 클래스 활용

- Collections 클래스
 - ▣ java.util 패키지에 포함
 - ▣ 컬렉션에 대해 연산을 수행하고 결과로 컬렉션 리턴
 - ▣ 모든 메소드는 static 타입
 - ▣ 주요 메소드
 - 컬렉션에 포함된 요소들을 소팅하는 sort() 메소드
 - 요소의 순서를 반대로 하는 reverse() 메소드
 - 요소들의 최대, 최솟값을 찾아내는 max(), min() 메소드
 - 특정 값을 검색하는 binarySearch() 메소드



Collections 클래스 활용

Collections 클래스를 활용하여 문자열 정렬, 반대로 정렬, 이진 검색 등을 실행하는 사례를 살펴보자.

```
import java.util.*;

public class CollectionsEx {
    static void printList(LinkedList<String> l) {
        Iterator<String> iterator = l.iterator();
        while (iterator.hasNext()) {
            String e = iterator.next();
            String separator;
            if (iterator.hasNext())
                separator = "->";
            else
                separator = "\n";
            System.out.print(e+separator);
        }
    }
}
```

```
public static void main(String[] args) {
    LinkedList<String> myList = new LinkedList<String>();
    myList.add("트랜스포머");
    myList.add("스타워즈");
    myList.add("매트릭스");
    myList.add(0,"터미네이터");
    myList.add(2,"아바타");

    Collections.sort(myList); // 요소 정렬
    printList(myList); // 정렬된 요소 출력

    Collections.reverse(myList); // 요소의 순서를 반대로
    printList(myList); // 요소 출력

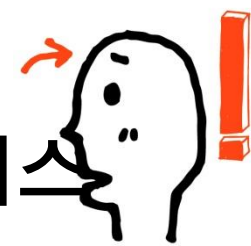
    int index = Collections.binarySearch(myList, "아바타") + 1;
    System.out.println("아바타는 " + index + "번째 요소입니다.");
}
```

static 메소드이므로
클래스 이름으로 바로 호출

소팅된 순서대로 출력

거꾸로 출력

매트릭스->스타워즈->아바타->터미네이터->트랜스포머
트랜스포머->터미네이터->아바타->스타워즈->매트릭스
아바타는 3번째 요소입니다.



Set<E> 인터페이스의 특성과 HashSet<E> 클래스

- List<E>를 구현하는 클래스들과 달리 Set<E>를 구현하는 클래스들은 데이터의 저장순서를 유지하지 않는다.
- List<E>를 구현하는 클래스들과 달리 Set<E>를 구현하는 클래스들은 데이터의 중복저장을 허용하지 않는다. 단, 동일 데이터에 대한 기준은 프로그래머가 정의
- 즉, Set<E>를 구현하는 클래스는 '집합'의 성격을 지닌다.

```
public static void main(String[] args)
{
    HashSet<String> hSet=new HashSet<String>();
    hSet.add("First");
    hSet.add("Second");
    hSet.add("Third");
    hSet.add("First");

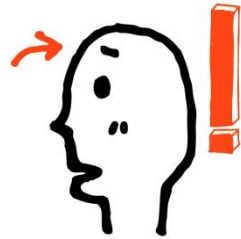
    System.out.println("저장된 데이터 수 : "+hSet.size());

    Iterator<String> itr=hSet.iterator();
    while(itr.hasNext())
        System.out.println(itr.next());
}
```

동일한 문자열 인스턴스는 저장
되지 않았다. 그렇다면 동일 인스
턴스를 판단하는 기준은?

실행 결과

저장된 데이터 수 : 3
Third
Second
First



동일 인스턴스에 대한 기준은?

```
public boolean equals(Object obj)
```

Object 클래스의 equals 메소드 호출 결과를 근거로 동일 인스턴스를 판단한다.

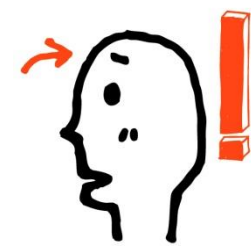
```
public int hashCode()
```

그런데 그에 앞서 Object 클래스의 hashCode 메소드 호출 결과가 같아야 한다.

정리하면,

두 인스턴스가 hashCode 메소드 호출 결과로 반환하는 값이 동일해야 한다.

그리고 이어서 두 인스턴스를 대상으로 equals 메소드의 호출 결과 true가 반환되면 동일 인스턴스로 간주한다.



동일 인스턴스의 판단기준 관찰을 위한 예

```
class SimpleNumber
{
    int num;
    public SimpleNumber(int n)
    {
        num=n;
    }
    public String toString()
    {
        return String.valueOf(num);
    }
}
```

HashSet<E> 클래스의 인스턴스 동등비교 방법

Object 클래스에 정의되어 있는 equals 메소드의 호출결과와 hashCode 메소드의 호출결과를 참조하여 인스턴스의 동등비교를 진행

```
public static void main(String[] args)
{
    HashSet<SimpleNumber> hSet=new HashSet<SimpleNumber>();
    hSet.add(new SimpleNumber(10));
    hSet.add(new SimpleNumber(20));
    hSet.add(new SimpleNumber(20));

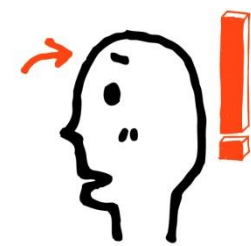
    System.out.println("저장된 데이터 수 : "+hSet.size());

    Iterator<SimpleNumber> itr=hSet.iterator();
    while(itr.hasNext())
        System.out.println(itr.next());
}
```

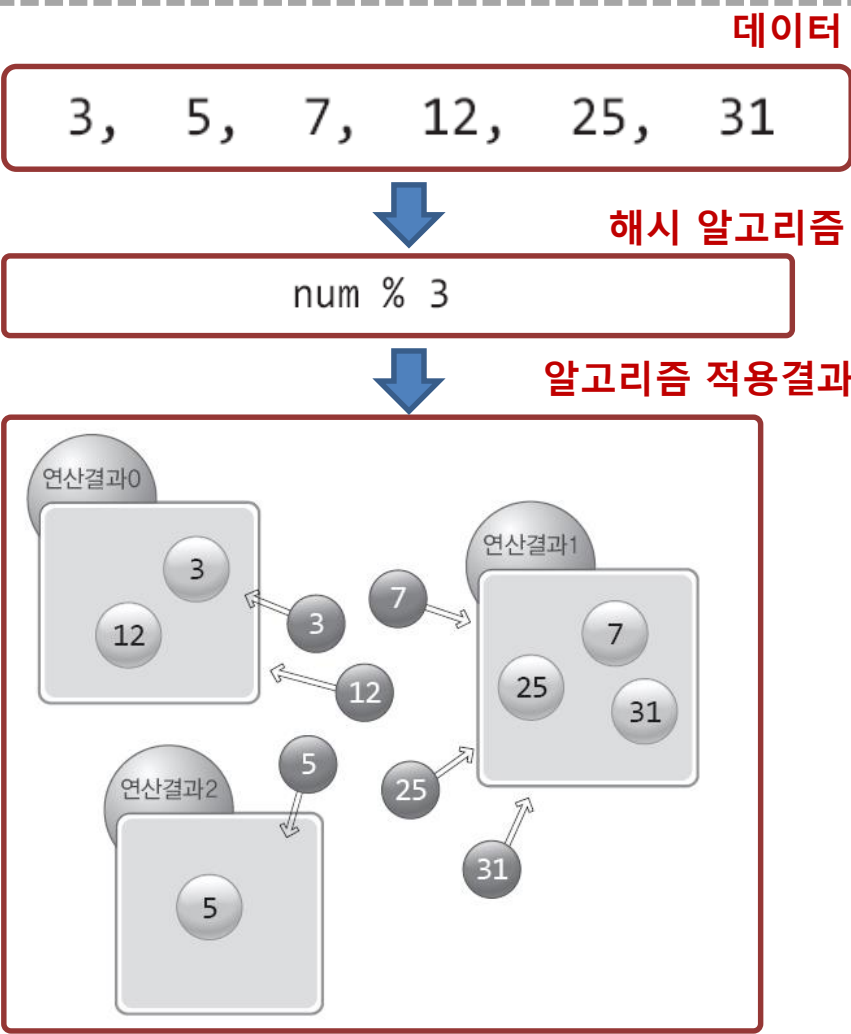
실행결과

저장된 데이터 수 : 3
20
10
20

실행결과를 보면, 동일 인스턴스의 판단기준이 별도로 존재함을 알 수 있다.



해시 알고리즘의 이해(데이터의 구분)



해시 알고리즘은 이렇듯 간단하게 디자인 될 수도 있다.

해시 알고리즘은 데이터의 분류에 사용이 된다. 데이터를 3으로 나머지 연산하였을때 얻게 되는 반환 값을 '해시 값'으로 하여 총 세 개의 부류를 구성하였다.

이렇게 분류해 놓으면, 데이터의 검색이 빨라진다. 정수 12가 저장되어있는지 확인한다고 했을때 문제 정수 12의 해시 값을 구한다. 그 다음에 해시 값에 해당하는 부류에서만 정수 12의 존재 유무를 확인하면 된다..



HashSet<E> 클래스의 동등비교

- 검색 1단계

Object 클래스의 hashCode 메소드의 반환 값을 해시 값으로 활용하여 검색의 그룹을 선택한다.

- 검색 2단계

그룹내의 인스턴스를 대상으로 Object 클래스의 equals 메소드의 반환 값의 결과로 동등을 판단

HashSet<E>의 인스턴스에 데이터의 저장을 명령하면, 우선 다음의 순서를 거치면서 동일 인스턴스가 저장되었는지를 확인한다.



따라서 아래의 두 메소드 적절히 오버라이딩 해야 함.

```
public int hashCode( )
public boolean equals(Object obj)
```

hashCode 메소드의 구현에 따라서 검색의 성능이 달라진다. 그리고 동일 인스턴스를 판단하는 기준이 맞게 equals 메소드를 정의해야 한다.



HashSet<E> 클래스의 동등비교

```
class SimpleNumber
{
    int num;
    . . . . .
    public int hashCode()
    {
        return num%3;
    }
    public boolean equals(Object obj)
    {
        SimpleNumber comp=(SimpleNumber)obj;
        if(comp.num==num)
            return true;
        else
            return false;
    }
}
```

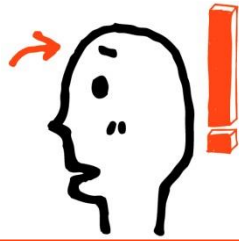
} %3 의 연산이 해시 알고리즘이니,
해시 그룹은 세 부류로 나뉜다.

} 인스턴스 변수 num이 같을때 동일
인스턴스로 간주하는 내용이 담겨
있다.

```
public static void main(String[] args)
{
    HashSet<SimpleNumber> hSet=new HashSet<SimpleNumber>();
    hSet.add(new SimpleNumber(10));
    hSet.add(new SimpleNumber(20));
    hSet.add(new SimpleNumber(20));
    System.out.println("저장된 데이터 수 : "+hSet.size());
    Iterator<SimpleNumber> itr=hSet.iterator();
    while(itr.hasNext())
        System.out.println(itr.next());
}
```

실행결과

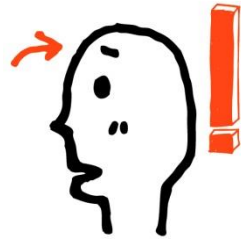
저장된 데이터 수 : 2
20
10



hashCode 메소드의 다양한 정의의 예

```
class Car {  
    private String model;  
    private String color;  
    . . . .  
  
    @Override  
    public int hashCode() {  
        return (model.hashCode() + color.hashCode()) / 2;  
    }  
}
```

. . . . 모든 인스턴스 변수의 정보를 다 반영하여 해쉬 값을 얻으려는 노력이 깃든 문장.
결과적으로 더 세밀하게 나뉘고, 따라서 그만큼 탐색 속도가 높아진다.



해쉬 알고리즘 일일이 정의하기 조금 그렇다면...

```
public static int hash(Object...values)
```

→ java.util.Objects에 정의된 메소드, 전달된 인자 기반의 해쉬 값 반환

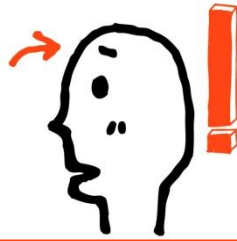
```
@Override
```

```
public int hashCode() {
```

```
    return Objects.hash(model, color);    // 전달인자 model, color 기반 해쉬 값 반환
```

```
}
```

전달된 인자를 모두 반영한 해쉬 값을 반환한다.

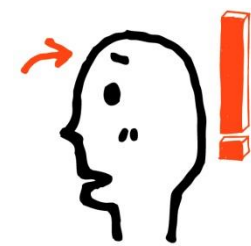


예제를 통한 HashSet<E>의 이해

▶ HashSet 구현 클래스

코드 7-7 package com.gilbut.chapter7;

```
1  import java.util.HashSet;
2  import java.util.Random;
3
4  public class HashSetExample1
5  {
6      private int count = 0;
7      private Random random = new Random();
8      private HashSet<Integer> set = new HashSet<Integer>();
9
10     public static void main(String[] args)
11     {
12         HashSetExample1 example = new HashSetExample1();
13         example.init();
14         example.execute();
15     }
16
```



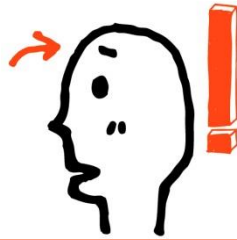
예제를 통한 HashSet<E>의 이해

▶ HashSet 구현 클래스

```
17 public void init()
18 {
19     // 10 미만의 random 정수를 클래스 변수 HashSet set에 입력
20     for (int i = 0; i < 10; i++)
21     {
22         set.add(random.nextInt(10));
23     }
24     this.printStatus(set);
25 }
26
27 public void execute()
28 {
29     HashSet<Integer> setObj = new HashSet<Integer>();
30     for (int i = 0; i < 10; i++)
31     {
32         //10 미만의 랜덤한 정수를 set 객체에 추가
33         setObj.add(random.nextInt(5));
34     }
35
36     this.printStatus(setObj);
37     set.addAll(setObj);
38     this.printStatus(set);
39 }
```

HashSet<Integer> set 객체는 중복된 값을 저장하지 않는다는 점을 기억하세요.

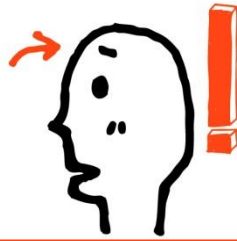
setObj 객체를 set 객체에 추가하면 결과가 어떻게 될까요?



예제를 통한 HashSet<E>의 이해

▶ HashSet 구현 클래스

```
40
41     public void printStatus(HashSet<Integer> hashSet)
42     {
43         if (hashSet == null || hashSet.size() == 0)
44         {
45             System.out.println("Object is null or size is zero");
46             return;
47         }
48
49         count++;
50
51         //배열로 내부 개수 확인
52         Integer[] array = hashSet.toArray(new Integer[hashSet.size()]);
53         System.out.print("count : " + count + ", ");
54         for (Integer item : array)
55         {
56             System.out.print "[" + item + " ] ";
57         }
58         System.out.print("\n");
59     }
60 }
```



예제를 통한 HashSet<E>의 이해

▶ HashSet 구현 클래스

실행결과

```
count : 1, [0] [1] [3] [5] [6] [7] [8]
count : 2, [0] [2] [3] [4]
count : 3, [0] [1] [2] [3] [4] [5] [6] [7] [8]
```

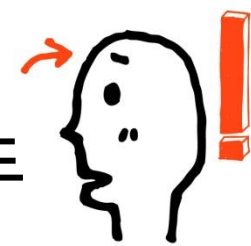
- ▶ 중복된 값을 허용하지 않는 것을 증명하기 위해서 두 개의 HashSet 객체를 사용
- ▶ HashSet 객체의 내부 멤버 객체를 화면에 출력하기 위해서 HashSet 객체를 배열로 변환하는 코드



HashMap<K, V> 클래스

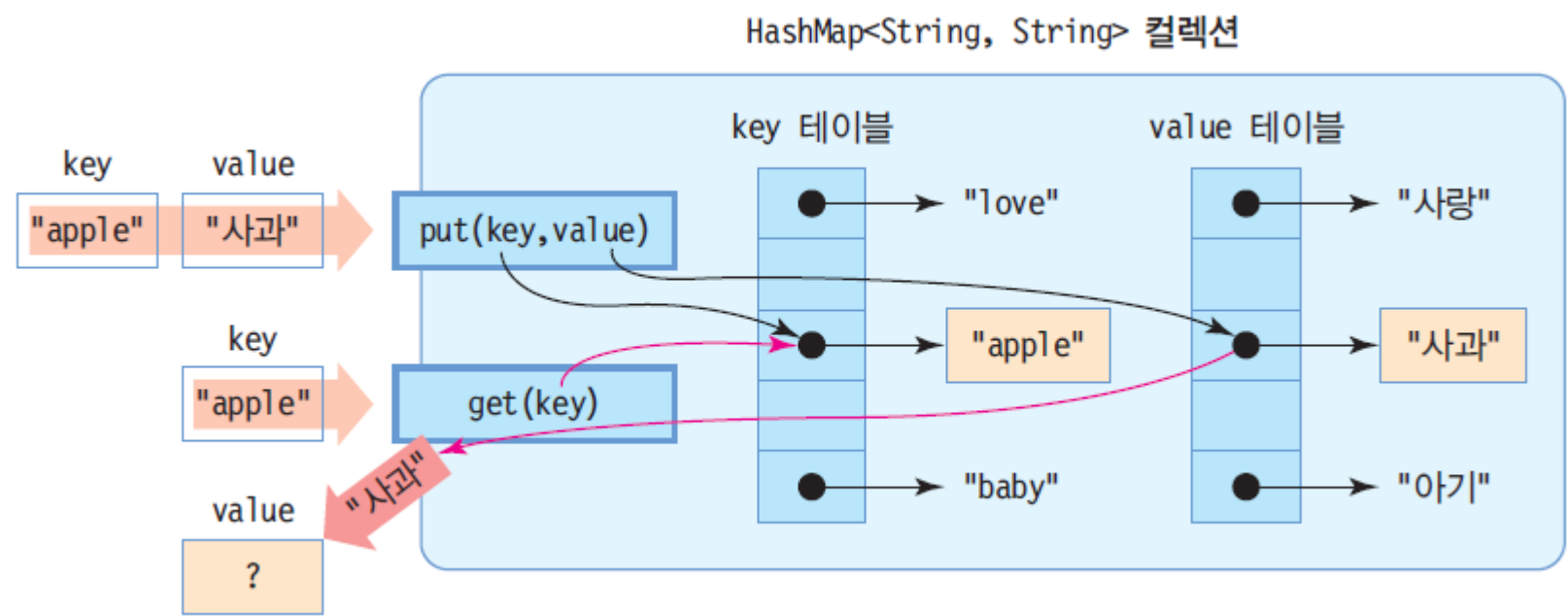
- HashMap<K,V>
 - 키(key)와 값(value)의 쌍으로 구성되는 요소를 다루는 컬렉션
 - java.util.HashMap
 - K는 키로 사용할 요소의 타입, V는 값으로 사용할 요소의 타입 지정
 - 키와 값이 한 쌍으로 삽입
 - 키는 해시맵에 삽입되는 위치 결정에 사용
 - 값을 검색하기 위해서는 반드시 키 이용
 - 삽입 및 검색이 빠른 특징
 - 요소 삽입 : put() 메소드
 - 요소 검색 : get() 메소드
 - 예) HashMap<String, String> 생성, 요소 삽입, 요소 검색

```
HashMap<String, String> h = new HashMap<String, String>();  
h.put("apple", "사과"); // "apple" 키와 "사과" 값의 쌍을 해시맵에 삽입  
String kor = h.get("apple"); // "apple" 키로 값 검색. kor는 "사과"
```



HashMap<String, String>의 내부 구성과 put(), get() 메소드

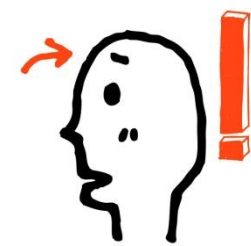
```
HashMap<String, String> map = new HashMap<String, String>();
```





HashMap<K,V>의 주요 메소드

메소드	설명
<code>void clear()</code>	HashMap의 모든 요소 삭제
<code>boolean containsKey(Object key)</code>	지정된 키(key)를 포함하고 있으면 <code>true</code> 리턴
<code>boolean containsValue(Object value)</code>	하나 이상의 키를 지정된 값(value)에 매핑시킬 수 있으면 <code>true</code> 리턴
<code>V get(Object key)</code>	지정된 키(key)에 매핑되는 값 리턴. 키에 매핑되는 어떤 값도 없으면 <code>null</code> 리턴
<code>boolean isEmpty()</code>	HashMap이 비어 있으면 <code>true</code> 리턴
<code>Set<K> keySet()</code>	HashMap에 있는 모든 키를 담은 <code>Set<K></code> 컬렉션 리턴
<code>V put(K key, V value)</code>	<code>key</code> 와 <code>value</code> 를 매핑하여 HashMap에 저장
<code>V remove(Object key)</code>	지정된 키(key)와 이에 매핑된 값을 HashMap에서 삭제
<code>int size()</code>	HashMap에 포함된 요소의 개수 리턴



HashMap<K,V>의 활용 예

해시맵 생성

```
HashMap<String, String> h =  
new HashMap<String, String>();
```

h

HashMap<String, String> 객체

key	value

(키, 값) 삽입

```
h.put("baby", "아기");  
h.put("love", "사랑");  
h.put("apple", "사과");
```

h

●	→	"love"	●	→	"사랑"
●	→	"apple"	●	→	"사과"
●	→	"baby"	●	→	"아기"

키로 값 읽기

```
String kor = h.get("love");
```

kor = "사랑"

키로 요소 삭제

```
h.remove("apple");
```

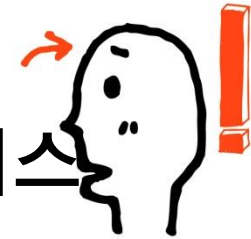
h

●	→	"love"	●	→	"사랑"
●	→	"baby"	●	→	"아기"

요소 개수

```
int n = h.size();
```

n = 2



Map<K, V> 인터페이스와 HashMap<K, V> 클래스

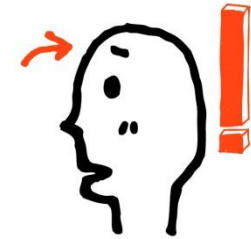
- Map<K, V> 인터페이스를 구현하는 컬렉션 클래스는 key-value 방식의 데이터 저장을 한다.
- value는 저장할 데이터를 의미하고, key는 value를 찾는 열쇠를 의미한다.
- Map<K, V>를 구현하는 대표적인 클래스로는 HashMap<K, V>와 TreeMap<K, V>가 있다.
- TreeMap<K, V>는 정렬된 형태로 데이터가 저장된다.

```
public static void main(String[] args)
{
    HashMap<Integer, String> hMap=new HashMap<Integer, String>();
    hMap.put(new Integer(3), "나삼번");
    hMap.put(5, "윤오번");
    hMap.put(8, "박팔번");
    System.out.println("6학년 3반 8번 학생 : "+hMap.get(new Integer(8)));
    System.out.println("6학년 3반 5번 학생 : "+hMap.get(5));
    System.out.println("6학년 3반 3번 학생 : "+hMap.get(3));

    hMap.remove(5);    // 5번 학생 전학 감
    System.out.println("6학년 3반 5번 학생 : "+hMap.get(5));
}
```

실행 결과

6학년 3반 8번 학생 : 박팔번
6학년 3반 5번 학생 : 윤오번
6학년 3반 3번 학생 : 나삼번
6학년 3반 5번 학생 : null



HashMap을 이용하여 영어 단어와 한글 단어를 쌍으로 저장하고 검색하는 사례

영어 단어와 한글 단어를 쌍으로 HashMap에 저장하고
영어 단어로 한글 단어를 검색하는 프로그램을 작성하라.

```
import java.util.*;

public class HashMapDicEx {
    public static void main(String[] args) {
        // 영어 단어와 한글 단어의 쌍을 저장하는 HashMap 컬렉션 생성
        HashMap<String, String> dic = new HashMap<String, String>();

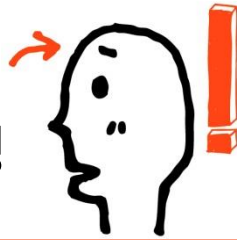
        // 3 개의 (key, value) 쌍을 dic에 저장
        dic.put("baby", "아기"); // "baby"는 key, "아기"은 value
        dic.put("love", "사랑");
        dic.put("apple", "사과");

        // dic 컬렉션에 들어 있는 모든 (key, value) 쌍 출력
        Set<String> keys = dic.keySet(); // key 문자열을 가진 Set 리턴
        Iterator<String> it = keys.iterator();
        while(it.hasNext()) {
            String key = it.next();
            String value = dic.get(key);
            System.out.println("(" + key + "," + value + ")");
        }
    }
}
```

```
// 영어 단어를 입력 받고 한글 단어 검색
Scanner scanner = new Scanner(System.in);
for(int i=0; i<3; i++) {
    System.out.print("찾고 싶은 단어는?");
    String eng = scanner.next();
    System.out.println(dic.get(eng));
}
}
```

```
(love,사랑)
(apple,사과)
(baby,아기)
찾고 싶은 단어는?apple
사과
찾고 싶은 단어는?babo
null
찾고 싶은 단어는?love
사랑
```

"babo"를 해시맵에서 찾을 수 없기 때문에 null 리턴



HashMap을 이용하여 자바 과목의 점수를 기록 관리하는 코드 작성

HashMap을 이용하여 학생의 이름과 자바 점수를 기록 관리해보자.

```
import java.util.*;

public class HashMapScoreEx {
    public static void main(String[] args) {
        // 사용자 이름과 점수를 기록하는 HashMap 컬렉션 생성
        HashMap<String, Integer> javaScore =
            new HashMap<String, Integer>();

        // 5 개의 점수 저장
        javaScore.put("한홍진", 97);
        javaScore.put("황기태", 34);
        javaScore.put("이영희", 98);
        javaScore.put("정원석", 70);
        javaScore.put("한원선", 99);

        System.out.println("HashMap의 요소 개수 : " + javaScore.size());

        // 모든 사람의 점수 출력.
        // javaScore에 들어 있는 모든 (key, value) 쌍 출력
        // key 문자열을 가진 집합 Set 컬렉션 리턴
        Set<String> keys = javaScore.keySet();

        // key 문자열을 순서대로 접근할 수 있는 Iterator 리턴
        Iterator<String> it = keys.iterator();
```

```
        while(it.hasNext()) {
            String name = it.next();
            int score = javaScore.get(name);
            System.out.println(name + " : " + score);
        }
    }
}
```

HashMap의 요소 개수 :5

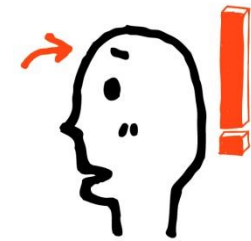
한원선 : 99

한홍진 : 97

황기태 : 34

이영희 : 98

정원석 : 70



HashMap을 이용한 학생 정보 저장

id와 전화번호로 구성되는 Student 클래스를 만들고, 이름을 '키' 로 하고 Student 객체를 '값' 으로 하는 해시맵을 작성하라.

```
import java.util.*;

class Student { // 학생을 표현하는 클래스
    int id;
    String tel;
    public Student(int id, String tel) {
        this.id = id; this.tel = tel;
    }
}
```

HashMap의 요소 개수 :3
한원선 : 2 010-222-2222
황기태 : 1 010-111-1111
이영희 : 3 010-333-3333

출력된 결과는 삽입된 결과와
다르다는 점을 기억하기 바람

```
public class HashMapStudentEx {
    public static void main(String[] args) {
        // 학생 이름과 Student 객체를 쌍으로 저장하는 HashMap 컬렉션 생성
        HashMap<String, Student> map = new HashMap<String, Student>();

        // 3 명의 학생 저장
        map.put("황기태", new Student(1, "010-111-1111"));
        map.put("한원선", new Student(2, "010-222-2222"));
        map.put("이영희", new Student(3, "010-333-3333"));

        System.out.println("HashMap의 요소 개수 : " + map.size());

        // 모든 학생 출력. map에 들어 있는 모든 (key, value) 쌍 출력
        // key 문자열을 가진 집합 Set 컬렉션 리턴
        Set<String> names = map.keySet();

        // key 문자열을 순서대로 접근할 수 있는 Iterator 리턴
        Iterator<String> it = names.iterator();
        while(it.hasNext()) {
            String name = it.next(); // 다음 키. 학생 이름
            Student student = map.get(name);
            System.out.println(name + " : " + student.id + " " + student.tel);
        }
    }
}
```



SelfCheck-1

- 1. 다음 클래스의 인스턴스가 HashSet<Person> 컬렉션 인스턴스에 저장 될 때, 이름과 나이가 같으면 동일 인스턴스로 판단이 되도록 hashCode와 equals 메소드를 오버라이딩 해보자

```
class Person {
    private String name;
    private int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String toString() {
        return name + "(" + age + "세)";
    }
}
```



SelfCheck-2

2. 다음과 같은 테이블 형태의 자료를 구조화하는 클래스를 만들고, 학번으로 검색 가능하도록 자바 자료구조(HashMap)를 이용하여 프로그래밍 하시오. 학번을 입력하면 해당 학생의 정보가 출력되도록 한다

학번	이름	전화번호	이메일
1001	홍길동	010-9999-9876	hong@mail.com
1002	아무개	010-8888-8765	anony@mail.com
1003	김무상	010-7777-7654	kim@mail.com

THANK YOU

실무에서 알아야 할 기술은 따로 있다! 자바를 다루는 기술